

Felipe Gallego Rengifo
Jorge Andrés López López

Ingeniería en sistemas

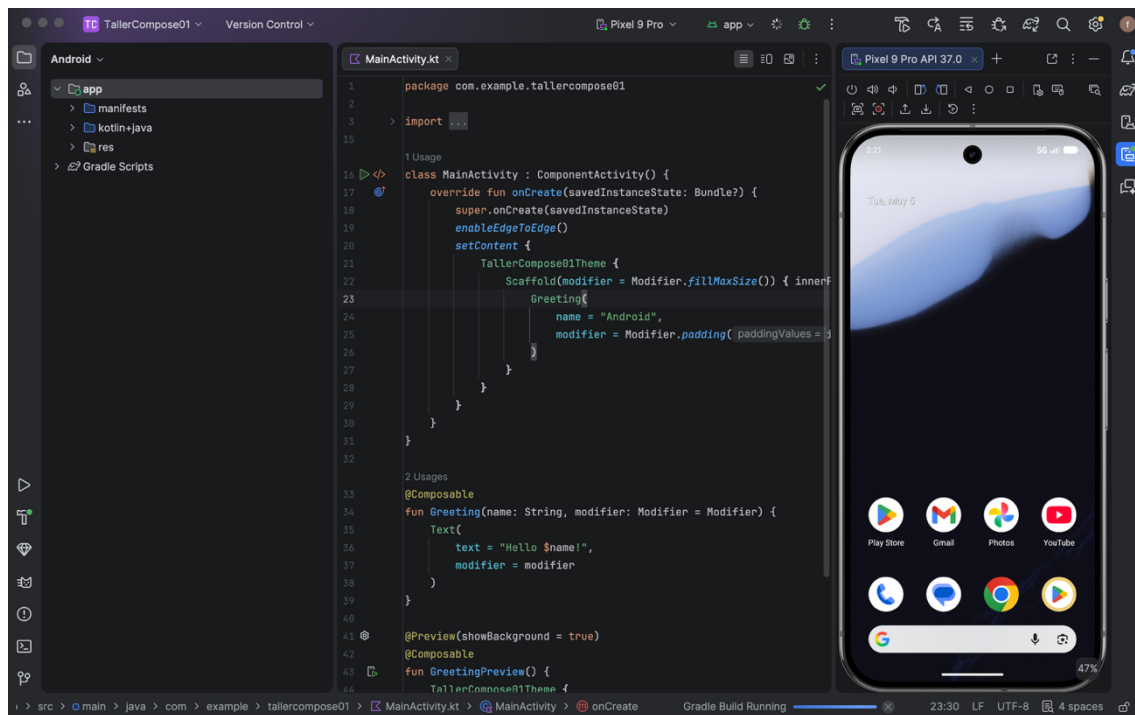
Taller de jetpack compas

Raul Gaviria

Universidad libre seccional Pereira

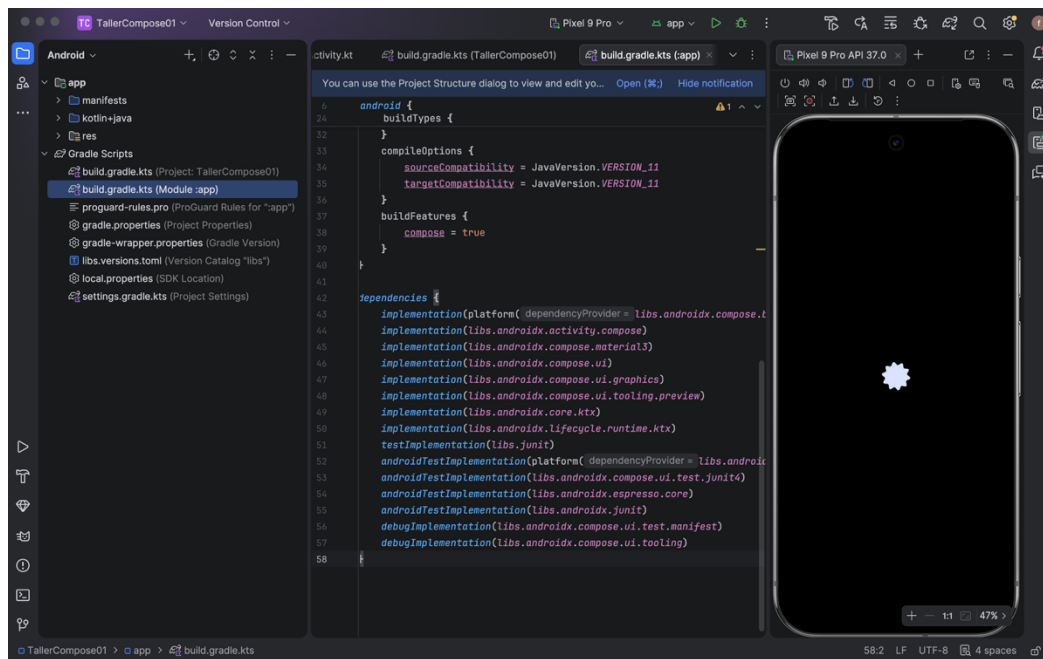
05/06/26

Crear el proyecto en Android Studio



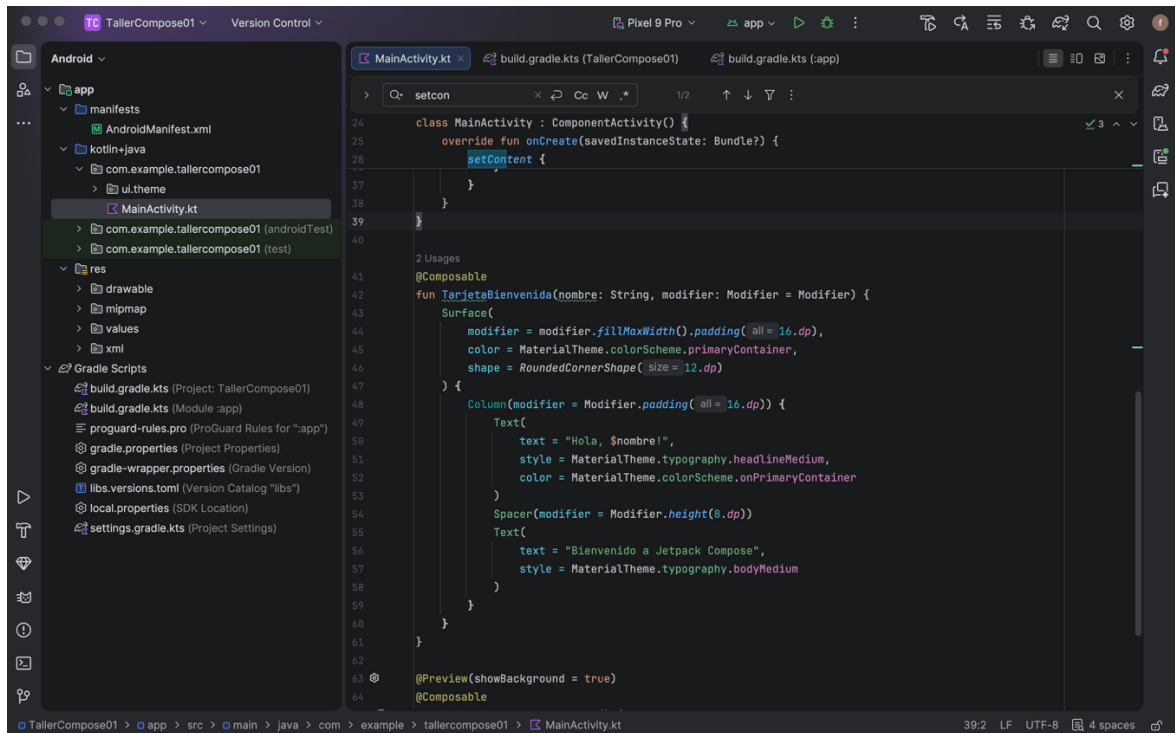
Se creó un nuevo proyecto en Android Studio seleccionando la plantilla Empty Activity, que incluye soporte nativo para Jetpack Compose. Se configuró el nombre del proyecto como TallerCompose01, con lenguaje Kotlin y Minimum SDK API 24. La plantilla genera automáticamente la estructura base con MainActivity.kt y el tema Material3 configurado.

Verificar las dependencias de Compose



Se verificó el archivo build.gradle.kts (Module: app) para confirmar que las dependencias de Jetpack Compose estaban incluidas correctamente. Al usar la plantilla moderna de Android Studio, las dependencias de androidx.compose.ui, androidx.compose.material3 y androidx.activity:activity-compose se generan automáticamente. El proyecto usa el sistema de catálogo de versiones libs.versions.toml para gestionar las versiones de forma centralizada.

Crear el Composable principal



Se implementó la función TarjetaBienvenida anotada con @Composable en el archivo MainActivity.kt. A continuación se describe cada componente utilizado:

@Composable

La anotación @Composable convierte una función Kotlin ordinaria en un elemento de interfaz de usuario que Jetpack Compose puede renderizar y gestionar. Toda función que construya UI debe tener esta anotación. El runtime de Compose gestiona cuándo re-ejecutar esa función de forma selectiva (proceso llamado recomposición) cada vez que el estado que lee cambia.

Scaffold

Scaffold es un Composable de Material3 que provee la estructura base de una pantalla, gestionando automáticamente el espaciado con innerPadding para que el contenido no quede oculto detrás de barras de navegación o de estado del sistema. Se usó como contenedor principal de la pantalla en el setContent {} de la MainActivity.

Surface

Surface es un Composable de Material3 que actúa como un contenedor visual con propiedades de diseño. Permite definir el color de fondo, la forma (en este caso esquinas redondeadas con RoundedCornerShape(12.dp)) y el tamaño mediante

modificadores. Es equivalente a un CardView en el sistema XML tradicional. Se usó como envoltorio principal de la tarjeta de bienvenida para darle apariencia visual consistente con el tema Material3.

Column

Column organiza sus elementos hijos en forma vertical, uno debajo del otro. Funciona de manera similar al LinearLayout con orientación vertical en XML. En este taller, Column contiene los dos textos de bienvenida y el espaciado entre ellos. Recibe un modifier con padding(16.dp) para que el contenido no quede pegado a los bordes de la Surface.

Text

Text es el Composable básico para mostrar texto en pantalla. Recibe parámetros como text (el contenido del string), style (el estilo tipográfico del sistema Material3 como headlineMedium y bodyMedium) y color (tomado del esquema de colores del tema). Se usaron dos instancias: una para el saludo con el nombre del usuario y otra para el mensaje de bienvenida.

Spacer

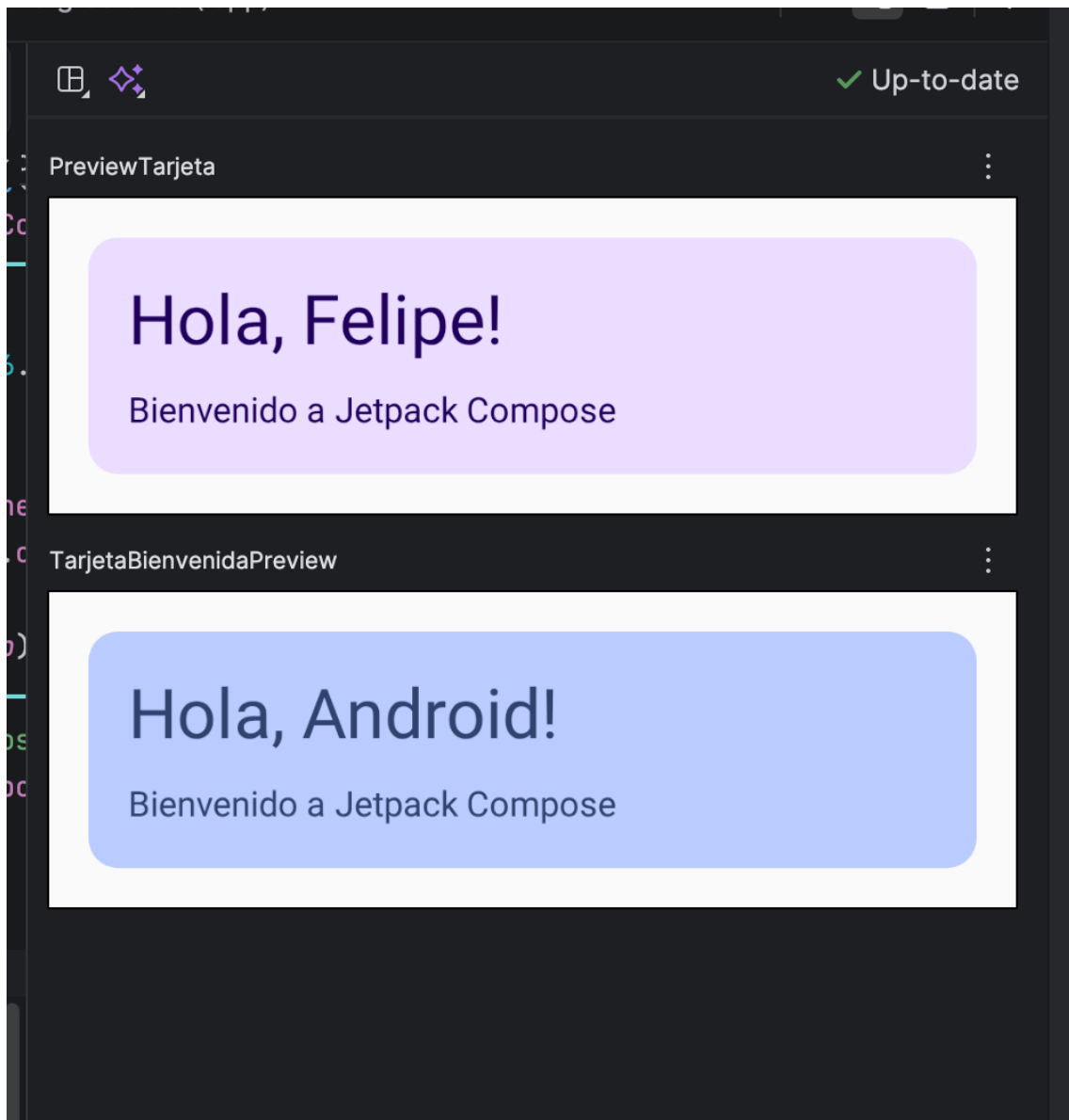
Spacer es un Composable vacío que sirve exclusivamente para agregar espacio entre otros elementos. No muestra nada visualmente, solo ocupa el espacio indicado en su modifier. Se usó Modifier.height(8.dp) para separar verticalmente los dos textos dentro de la Column. Es equivalente a usar margin en XML pero de forma explícita y declarativa.

Modifier

Modifier no es un Composable por sí solo, sino una cadena de instrucciones de diseño que se aplica a cualquier Composable. Permite encadenar ajustes como .fillMaxWidth() (ocupa todo el ancho disponible), .padding(16.dp) (agrega espacio interior o exterior) y .height(8.dp) (define una altura fija). En Jetpack Compose, Modifier reemplaza múltiples atributos XML como layout_width, layout_margin y padding, concentrándolos en una sola cadena de código legible.

Agregar el @Preview

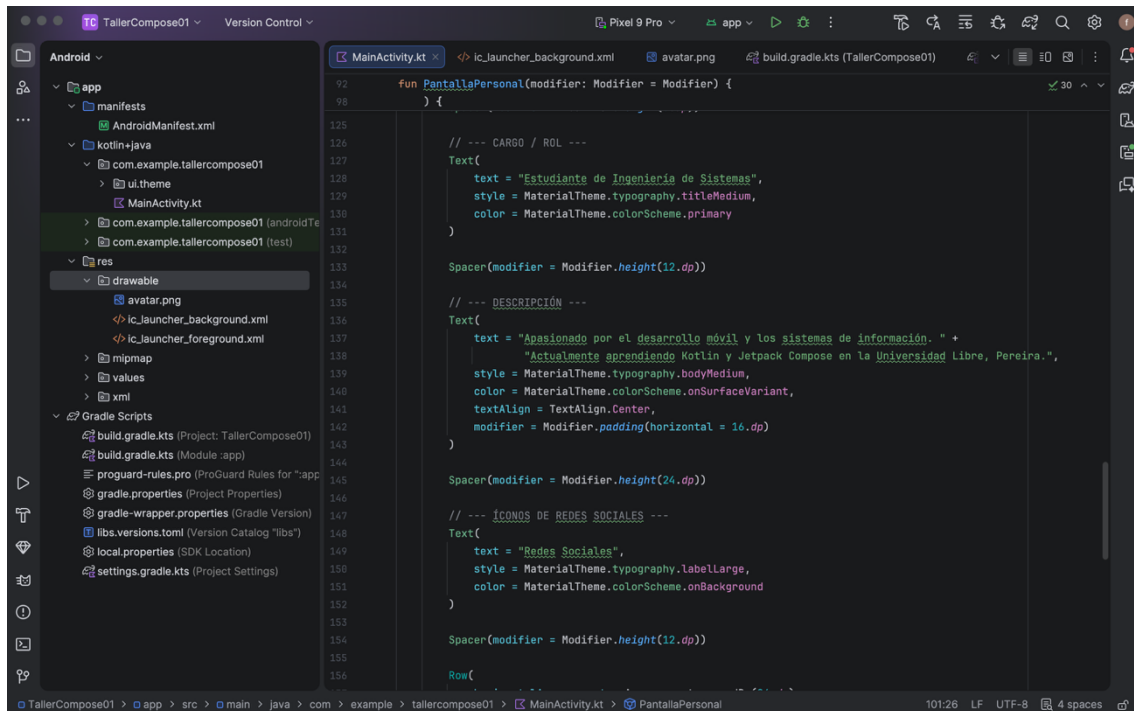
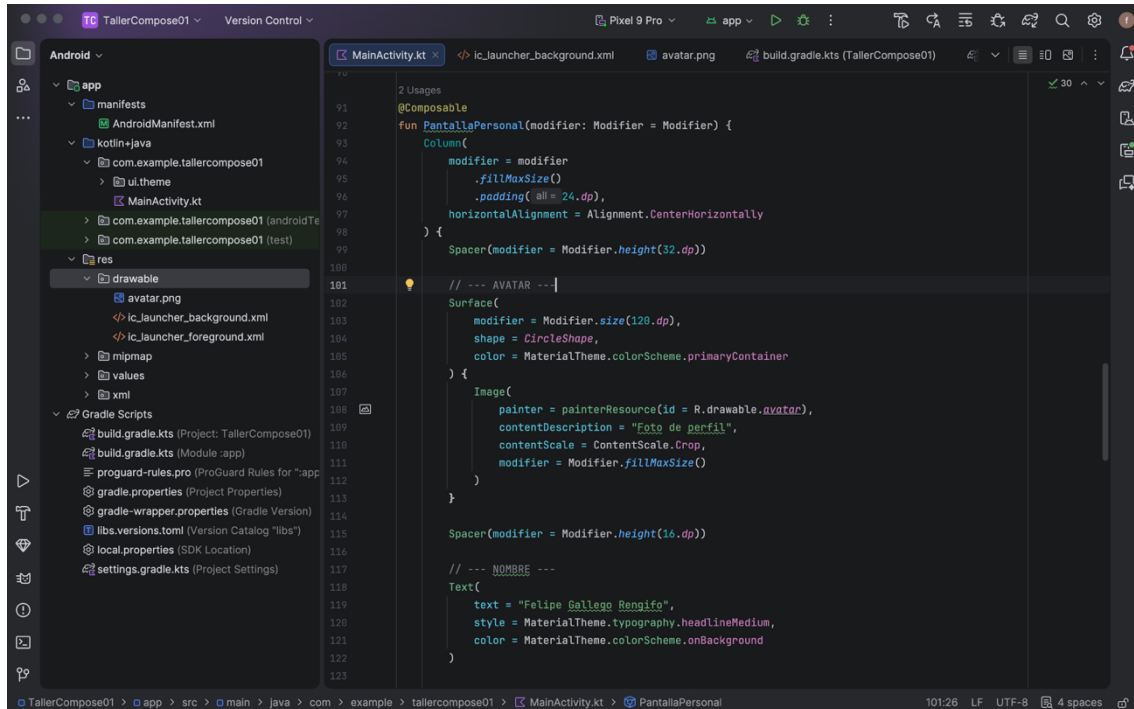
```
@Preview(showBackground = true)
@Composable
fun PreviewTarjeta() {
    TarjetaBienvenida(nombre = "Felipe")
}
```

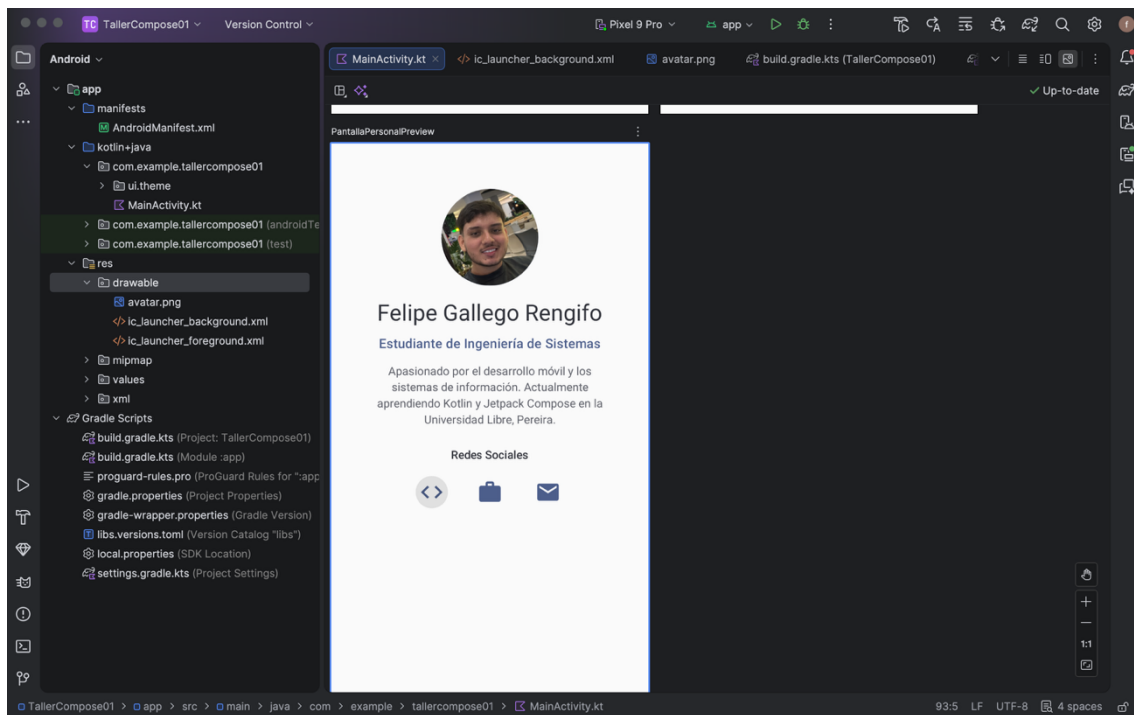
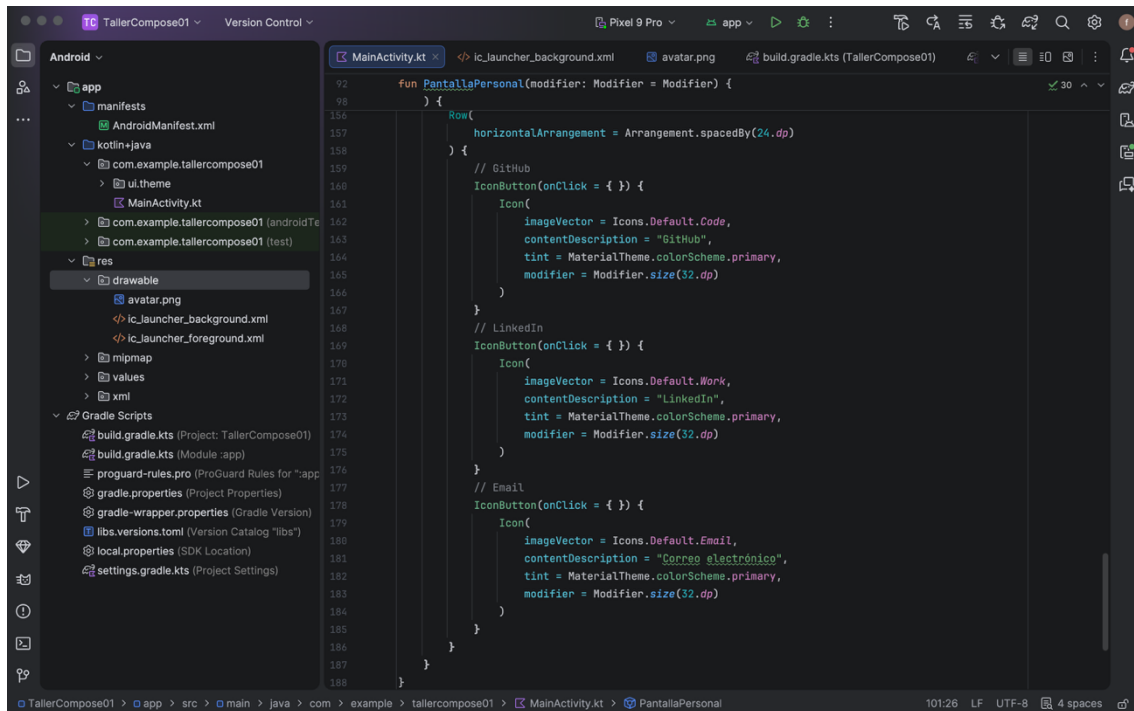


Se agregó la función `TarjetaBienvenidaPreview` anotada con `@Preview(showBackground = true)` para visualizar el Composable directamente en Android Studio sin necesidad de compilar ni lanzar el emulador. Está envuelta en

TallerCompose01Theme para que la previsualización respete los colores y tipografía del tema Material3. Se usó la vista Split del editor para ver el código y la previsualización simultáneamente, lo que permite iterar el diseño de forma ágil.

Actividad de Extensión



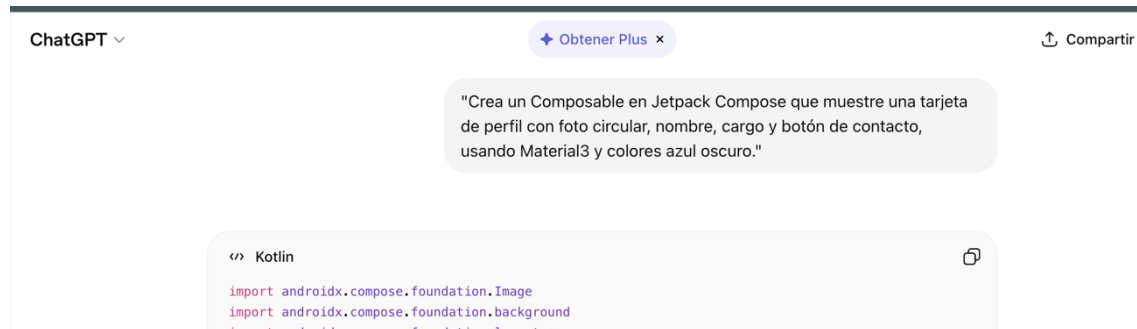


Se creó una pantalla de presentación personal implementando un Column centrado horizontalmente con Alignment.CenterHorizontally. La pantalla incluye un avatar en forma circular usando Surface con CircleShape, el nombre completo del estudiante, el rol o cargo, una descripción personal y tres íconos de redes sociales organizados en un Row. Los íconos representan GitHub (Icons.Default.Code), LinkedIn (Icons.Default.Work) y correo electrónico (Icons.Default.Email), cada uno envuelto en un IconButton para que sea

interactivo. Todos los colores provienen del sistema `MaterialTheme.colorScheme` para garantizar consistencia con el tema `Material3`.

Actividad con IA

Se utilizó la herramienta ChatGPT con el siguiente prompt:



¿Qué Composables usó la IA?

La IA utilizó varios Composables de Jetpack Compose como `Column`, `Row`, `Text`, `Image`, `Surface`, `Button`, `Spacer`, `IconButton` e `Icon`. También usó componentes de `Material3` para aplicar estilos modernos y colores personalizados. Es destacable que la IA organizó la jerarquía de Composables de forma lógica, colocando el avatar en un `Surface` con `CircleShape` para lograr la forma circular, y agrupando los botones en un `Row` para distribuirlos horizontalmente.

¿Qué partes tuviste que corregir o personalizar?

Fue necesario personalizar el `drawable` para agregar la imagen de perfil (`avatar.png`) dentro de la carpeta `res/drawable`. Además, se ajustaron los colores azul oscuro y detalles visuales para que el diseño se viera más acorde al estilo del proyecto. También fue necesario corregir algunos imports que la IA generó de forma incorrecta, ya que usaba rutas de paquetes desactualizadas incompatibles con la versión de Compose del proyecto. El nombre del parámetro en el Composable también debió ajustarse para seguir la convención de nomenclatura establecida.

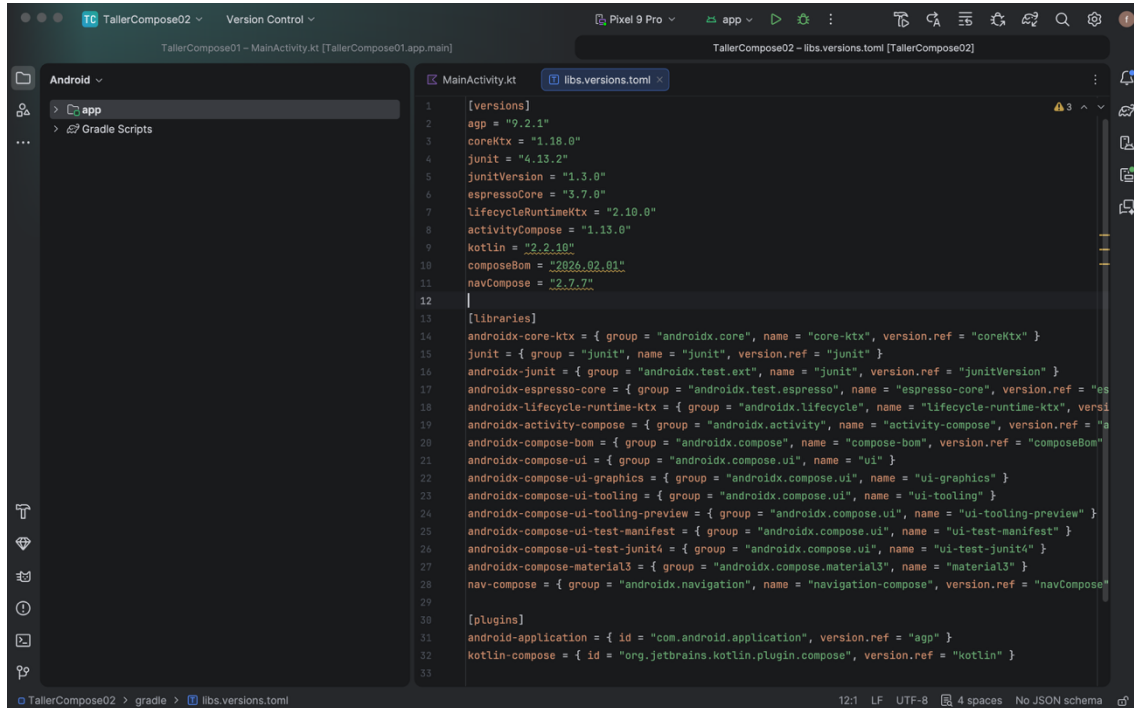
¿Por qué crees que la IA puede equivocarse en código?

La IA puede equivocarse porque genera código basándose en patrones estadísticos aprendidos durante su entrenamiento, sin tener acceso real al proyecto ni conocer las versiones exactas de las librerías instaladas. Por ejemplo, puede usar imports incorrectos, nombres de archivos que no existen en `res/drawable`, o componentes incompatibles con la versión de `Material3` configurada en el proyecto. Además, la IA no ejecuta el código que genera, por lo que no detecta errores en tiempo de compilación. Esto refuerza la importancia de entender el código generado antes de usarlo, revisarlo críticamente y

personalizarlo según las necesidades reales del proyecto, tal como establece la política del curso.

TALLER 02 — Estado y Navegación: App de Lista de Tareas

Agregar la dependencia de Navigation



```
implementation(libs.androidx.lifecycle.livedata)
implementation(libs.nav.compose)
testImplementation(libs.junit)
```

Se agregó la dependencia navigation-compose al proyecto para habilitar la navegación entre pantallas de forma declarativa. Se configuró en el archivo libs.versions.toml la versión de la librería y se referenció en build.gradle.kts (Module: app). Después de sincronizar el proyecto con Sync Now, la dependencia quedó disponible para su uso en el código.

Crear la data class Tarea



```
1 package com.example.tallercompose02
2
3 data class Tarea(
4     val id: Int,
5     val titulo: String,
6     val completada: Boolean = false
7 )
```

Se definió la clase de datos Tarea con tres campos: id de tipo Int como identificador único, titulo de tipo String para el nombre de la tarea, y completada de tipo Boolean con valor por defecto false. Al ser una data class de Kotlin, genera automáticamente los métodos equals(), hashCode(), copy() y toString(). El método copy() es especialmente importante en Compose porque permite crear una nueva instancia modificando solo un campo, respetando el principio de inmutabilidad del estado reactivo.

El código completo del MainActivity.kt

```
// --- NAVEGACIÓN ---
1 Usage
@Composable
fun AppNavegacion() {
    val navController = rememberNavController()
    var tareas by remember { mutableStateOf( value = ListOf<Tarea>()) }

    NavHost(navController = navController, startDestination = "lista") {
        composable( route = "lista") {
            ListaTareasScreen(
                tareas = tareas,
                onAgregarTarea = { nueva -> tareas = tareas + nueva },
                onEliminarTarea = { id -> tareas = tareas.filter { it.id != id } },
                onCambiarEstado = { id, checked ->
                    tareas = tareas.map { if (it.id == id) it.copy(completada = checked) else it },
                },
                navController = navController
            )
        }
        composable( route = "detalle/{tareaId}") { backStackEntry ->
            val id = backStackEntry.arguments?.getString( key = "tareaId")?.toIntOrNull()
            val tarea = tareas.find { it.id == id }
            if (tarea != null) {
                DetalleTareaScreen(tarea = tarea, navController = navController)
            }
        }
    }
}
}
```

```
// --- PANTALLA LISTA ---
1 Usage
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun ListaTareasScreen(
    tareas: List<Tarea>,
    onAgregarTarea: (Tarea) -> Unit,
    onEliminarTarea: (Int) -> Unit,
    onCambiarEstado: (Int, Boolean) -> Unit,
    navController: NavController
) {
    var mostrarDialogo by remember { mutableStateOf( value = false) }

    Scaffold(
        topBar = { TopAppBar(title = { Text("Lista de Tareas") }) },
        floatingActionButton = {
            FloatingActionButton(onClick = { mostrarDialogo = true }) {
                Icon( ImageVector = Icons.Default.Add, contentDescription = "Agregar tarea")
            }
        }
    ) { innerPadding ->
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding( paddingValues = innerPadding)
                .padding( all = 16.dp)
        ) {
            items(tareas, key = { it.id }) { tarea ->
                TareaItem(
                    tarea = tarea,
                    onCheckedChange = { checked -> onCambiarEstado(tarea.id, checked) },
                    onEliminar = { onEliminarTarea(tarea.id) },
                    onClick = { navController.navigate( route = "detalle/${tarea.id}") }
                )
            }
        }
    }
}
```



```
// — ITEM DE TAREA
1 Usage
@Composable
fun TareaItem(
    tarea: Tarea,
    onCheckedChange: (Boolean) -> Unit,
    onEliminar: () -> Unit,
    onClick: () -> Unit
) {
    Card(
        onClick = onClick,
        modifier = Modifier.fillMaxWidth()
    ) {
        Row(
            modifier = Modifier.padding(all = 12.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Checkbox(
                checked = tarea.completada,
                onCheckedChange = onCheckedChange
            )
            Text(
                text = tarea.titulo,
                modifier = Modifier.weight(1f).padding(horizontal = 8.dp),
                textDecoration = if (tarea.completada) TextDecoration.LineThrough else null,
                color = if (tarea.completada)
                    MaterialTheme.colorScheme.onSurface.copy(alpha = 0.4f)
                else
                    MaterialTheme.colorScheme.onSurface
            )
            IconButton(onClick = onEliminar) {
                Icon(ImageVector = Icons.Default.Delete, contentDescription = "Eliminar")
            }
        }
    }
}
```

```
// — DIÁLOGO NUEVA TAREA
1 Usage
@Composable
fun DialogoNuevaTarea(onConfirmar: (String) -> Unit, onCancel: () -> Unit) {
    var texto by remember { mutableStateOf(value = "") }
    var error by remember { mutableStateOf(value = false) }

    AlertDialog(
        onDismissRequest = onCancel,
        title = { Text("Nueva tarea") },
        text = {
            Column {
                OutlinedTextField(
                    value = texto,
                    onValueChange = {
                        texto = it
                        error = false
                    },
                    label = { Text("Título de la tarea") },
                    isError = error,
                    supportingText = {
                        if (error) Text(
                            "El título no puede estar vacío",
                            color = MaterialTheme.colorScheme.error
                        )
                    }
                )
            }
        },
        confirmButton = {

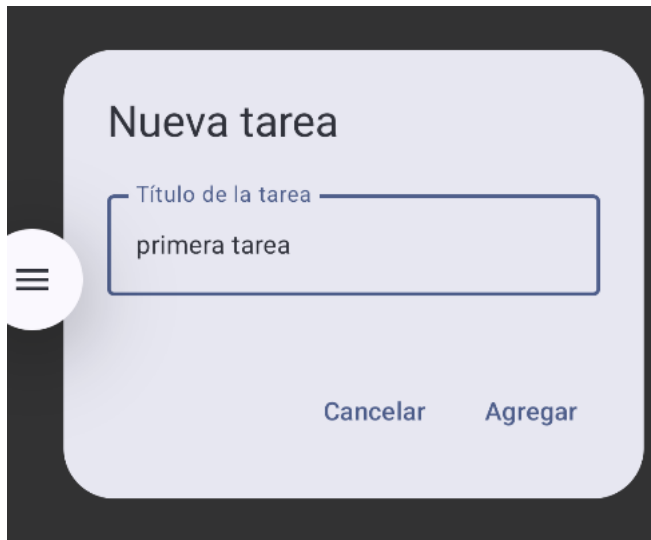
```

```
// — PANTALLA DETALLE —
1 Usage
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun DetalleTareaScreen(tarea: Tarea, navController: NavController) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Detalle de Tarea") },
                navigationIcon = {
                    TextButton(onClick = { navController.popBackStack() }) {
                        Text("← Volver")
                    }
                }
            )
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding( paddingValues = innerPadding)
                .padding( all = 24.dp)
        ) {
            Text("Tarea #${tarea.id}", style = MaterialTheme.typography.labelLarge)
            Spacer(modifier = Modifier.height(8.dp))
            Text(tarea.titulo, style = MaterialTheme.typography.headlineMedium)
            Spacer(modifier = Modifier.height(16.dp))
            Text(
                text = if (tarea.completada) "✅ Completada" else "❌ Pendiente",
                style = MaterialTheme.typography.bodyLarge,
            )
        }
    }
}
```

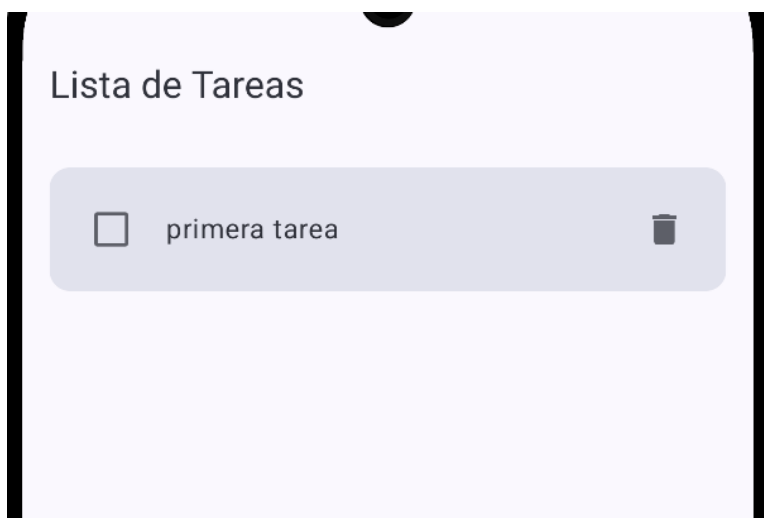
Ejecutar y probar



El estado de la aplicación se gestiona con `remember { mutableStateOf() }`, que permite a Compose detectar cambios y re-ejecutar automáticamente los Composables afectados. La lista de tareas se declara como `var tareas by remember { mutableStateOf(listOf<Tarea>()) }`. Cada operación (agregar, eliminar, cambiar estado) genera una nueva lista en lugar de modificar la existente, lo que garantiza que Compose detecte el cambio y actualice la UI correctamente. Este patrón de inmutabilidad es fundamental en la programación declarativa.

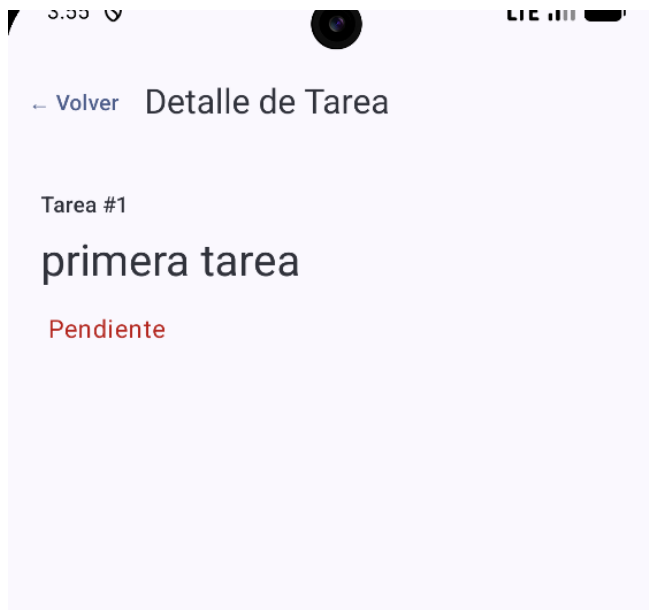
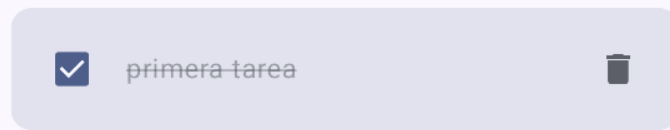


Se implementó la navegación entre pantallas usando `NavHost` y `NavController` de `Navigation Compose`. El `NavHost` define dos rutas: "lista" como destino inicial y "detalle/{tareaid}" como destino secundario. El id de la tarea se pasa como argumento en la URL de navegación con `navController.navigate("detalle/${tarea.id}")`, y se recupera en la pantalla de detalle con `backStackEntry.arguments?.getString("tareaid")`. El botón de regreso usa `navController.popBackStack()` para volver al destino anterior.



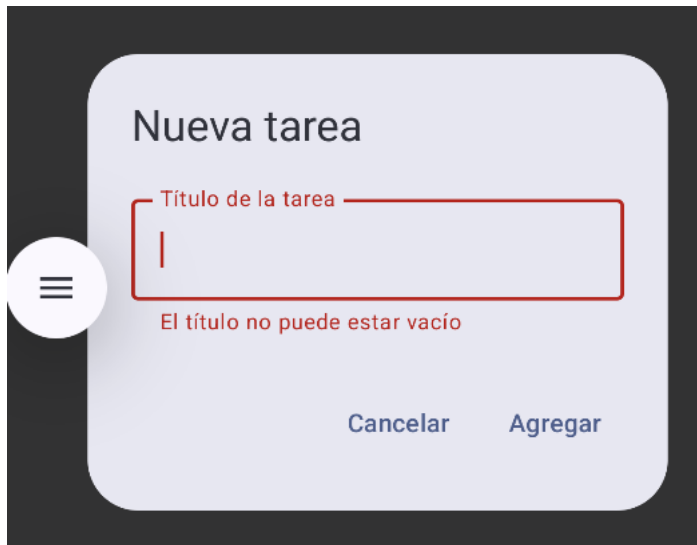
Se implementó `LazyColumn` para renderizar la lista de tareas de forma eficiente. A diferencia de un `Column` normal que renderiza todos los elementos, `LazyColumn` solo renderiza los ítems visibles en pantalla, similar al `RecyclerView` del sistema XML. Se usa el parámetro `key = { it.id }` para identificar cada ítem de forma única, lo que permite a `Compose` optimizar las recomposiciones y aplicar animaciones correctamente cuando se agregan o eliminan elementos.

Lista de Tareas



Se implementó la navegación entre pantallas usando NavHost y NavController de Navigation Compose. El NavHost define dos rutas: "lista" como destino inicial y "detalle/{tareaid}" como destino secundario. El id de la tarea se pasa como argumento en la URL de navegación con `navController.navigate("detalle/${tarea.id}")`, y se recupera en la pantalla de detalle con `backStackEntry.arguments?.getString("tareaid")`. El botón de regreso

usa `navController.popBackStack()` para volver al destino anterior.



Se implementó validación de entrada en el `DialogoNuevaTarea` usando un `OutlinedTextField` con el parámetro `isError`. Cuando el usuario intenta agregar una tarea con el campo vacío, la variable `error` cambia a `true`, lo que activa el borde rojo del campo y muestra el mensaje "El título no puede estar vacío" mediante el parámetro `supportingText`. El error se limpia automáticamente cuando el usuario comienza a escribir, gracias al `onValueChange` que resetea `error = false`.

Actividad con IA

Se utilizó ChatGPT con el siguiente prompt

Crea un Composable `TareaItem` en Jetpack Compose con `Checkbox`, texto tachado si está completada, ícono de eliminar a la derecha y animación `fade` al marcar como hecha. Material3."

<> Kotlin

```
@Composable
fun TareaItem(
    tarea: String,
```

¿Qué Composables usó la IA?

La IA generó un Composable `TareaItem` usando `Row` como contenedor principal para alinear horizontalmente el `Checkbox`, el `Text` y el `IconButton` con el ícono de eliminar. Para el texto tachado usó el parámetro `textDecoration = TextDecoration.LineThrough` del Composable `Text`. Para la animación de `fade` implementó `animateFloatAsState` aplicado al `alpha` del `Modifier`, lo que produce una transición suave de opacidad cuando la tarea se marca como completada. También usó `Card` como contenedor externo para dar elevación visual al ítem.

¿Qué partes tuviste que corregir o personalizar?

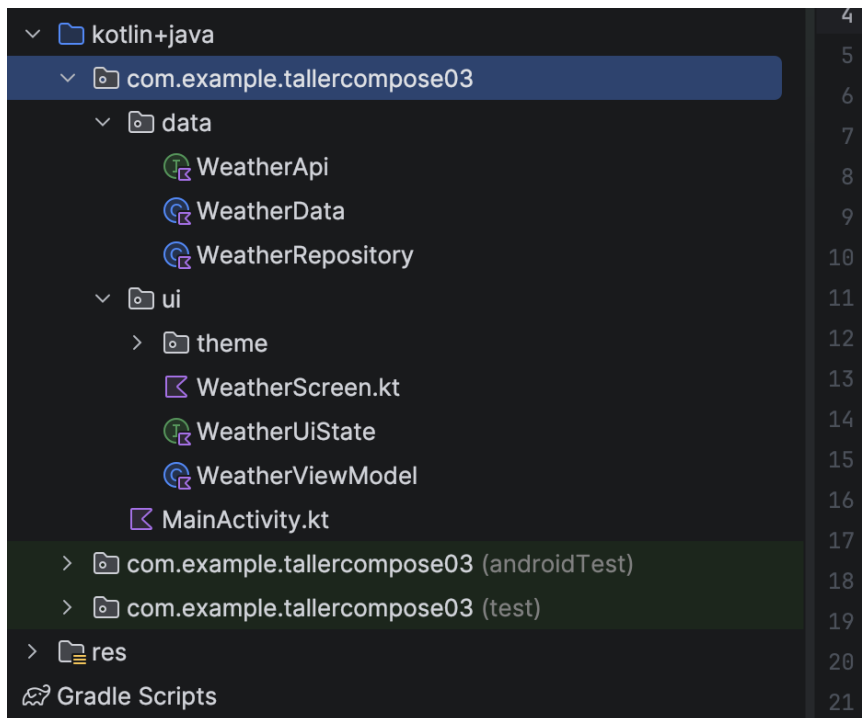
Fue necesario ajustar los colores del texto al usar `MaterialTheme.colorScheme.onSurface.copy(alpha = 0.4f)` para el estado completado, ya que la IA usaba colores hardcodedos que no respetaban el tema `Material3`. También se corrigió la firma de la función para recibir el objeto `Tarea` completo en lugar de parámetros individuales, lo que facilita su integración con el `LazyColumn`. Además, se eliminó una importación duplicada que la IA incluyó y que generaba un error de compilación.

¿Por qué crees que la IA puede equivocarse en código?

La IA puede generar código incorrecto porque no tiene acceso al contexto real del proyecto: desconoce las versiones exactas de las librerías, la estructura de los archivos y las convenciones de nomenclatura adoptadas. En este caso, la animación `animateFloatAsState` requería importar correctamente `androidx.compose.animation.core`, algo que la IA omitió. Esto demuestra que el código generado por IA siempre debe ser revisado críticamente, probado en el proyecto real y ajustado según sea necesario, sin asumirlo como correcto de forma automática. La IA es una herramienta de apoyo, no un reemplazo del criterio del desarrollador.

Taller 03 App del Clima con API REST — ViewModel + StateFlow

Arquitectura MVVM del Proyecto



Se implementó la arquitectura MVVM (Model-View-ViewModel) que separa la lógica de negocio de la interfaz de usuario en capas independientes. La capa data/ contiene las clases de datos (WeatherData), la interfaz Retrofit (WeatherApi) y el repositorio (WeatherRepository). La capa ui/ contiene el estado de UI (WeatherUiState), el ViewModel (WeatherViewModel) y la pantalla Composable (WeatherScreen). Esta separación garantiza que la UI solo se encargue de mostrar datos y que el ViewModel nunca tenga referencias directas a Composables o Contextos de Android.

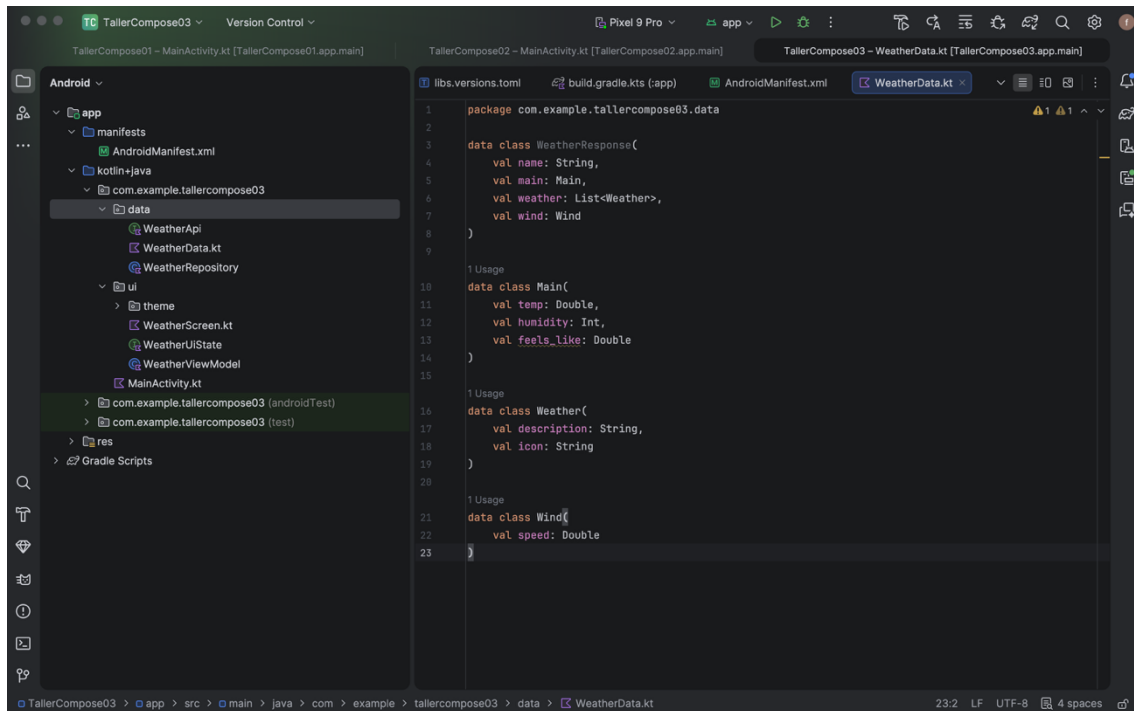
Dependencias configuradas

```
[versions]
agp = "9.2.1"
coreKtx = "1.10.0"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
lifecycleRuntimeKtx = "2.10.0"
activityCompose = "1.13.0"
kotlin = "2.2.10"
composeBom = "2026.02.01"
retrofit = "2.9.0"
coroutines = "1.7.3"
lifecycle = "2.7.0"
hilt = "2.50"
materialIcons = "1.7.5"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }
retrofit-core = { group = "com.squareup.retrofit2", name = "retrofit", version.ref = "retrofit" }
```


Se configuraron las dependencias necesarias en el archivo `libs.versions.toml` siguiendo el sistema de catálogo de versiones de Gradle. Las librerías añadidas fueron: `retrofit2` para el cliente HTTP, `converter-gson` para deserializar JSON automáticamente, `kotlinx-coroutines-android` para operaciones asíncronas, y `lifecycle-viewmodel-compose` con `lifecycle-runtime-compose` para observar el estado del ViewModel desde Composables. El permiso de Internet se agregó en `AndroidManifest.xml` con la etiqueta `<uses-permission android:name="android.permission.INTERNET"/>` para habilitar las llamadas de red.

Modelo de Datos (WeatherData.kt)



Se definieron las data classes que modelan la respuesta JSON de OpenWeatherMap. La clase principal `WeatherResponse` contiene el nombre de la ciudad (`name`), un objeto `Main` con temperatura, humedad y sensación térmica, una lista `Weather` con la descripción e ícono del clima, y un objeto `Wind` con la velocidad del viento. Estas clases son deserializadas automáticamente por la librería `Gson` de `Retrofit`, mapeando cada campo del JSON al atributo Kotlin correspondiente por coincidencia de nombre.

Interfaz Retrofit y Repositorio

```
build.gradle.kts (:app)  AndroidManifest.xml  WeatherData.kt  WeatherApi.kt x
package com.example.tallercompose03.data

import retrofit2.http.GET
import retrofit2.http.Query

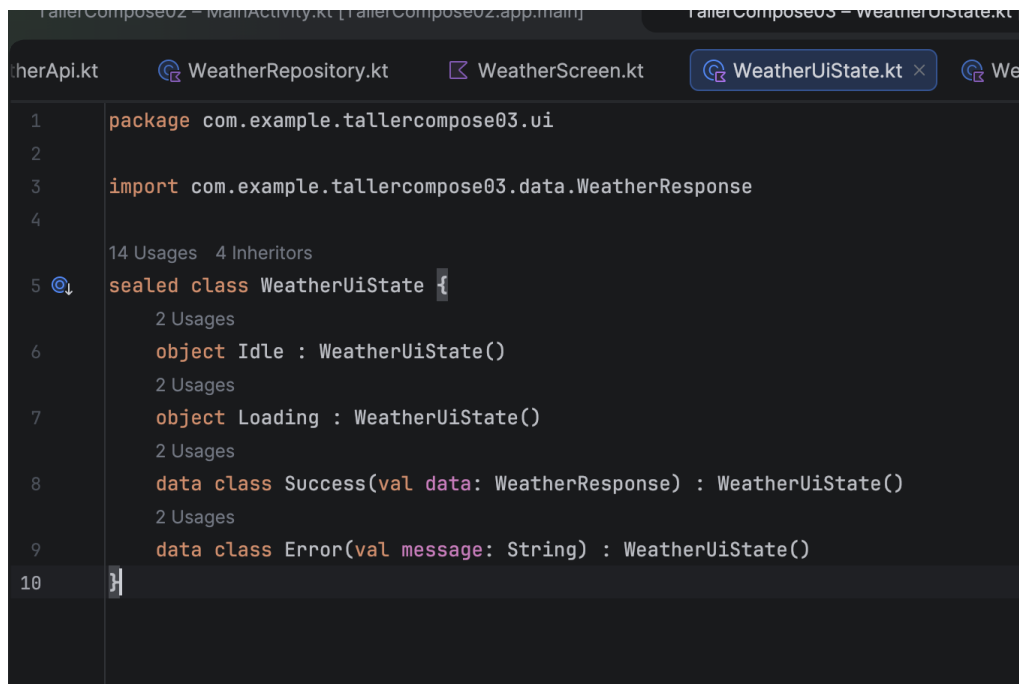
1 Usage
interface WeatherApi {
    1 Usage
    @GET(value = "weather")
    suspend fun getWeather(
        @Query(value = "q") city: String,
        @Query(value = "appid") apiKey: String,
        @Query(value = "units") units: String = "metric",
        @Query(value = "lang") lang: String = "es"
    ): WeatherResponse
}
```

```
AndroidManifest.xml  WeatherData.kt  WeatherApi.kt  WeatherRepository.kt x  Weather
1 package com.example.tallercompose03.data
2
3 import retrofit2.Retrofit
4 import retrofit2.converter.gson.GsonConverterFactory
5
6 2 Usages
class WeatherRepository {
    1 Usage
    private val api = Retrofit.Builder()
        .baseUrl("https://api.openweathermap.org/data/2.5/")
        .addConverterFactory(GsonConverterFactory.create())
        .build()
        .create(WeatherApi::class.java)

    1 Usage
    suspend fun getWeather(city: String): WeatherResponse {
14 → return api.getWeather(city = city, apiKey = "ae0e1514db663baa35296842ec998602")
15 }
16 }
```

La interfaz WeatherApi define el contrato de la llamada HTTP con la anotación @GET("weather") y parámetros @Query para la ciudad, la API Key, las unidades métricas y el idioma. La función está marcada con suspend para ejecutarse dentro de una Coroutine sin bloquear el hilo principal. El WeatherRepository construye la instancia de Retrofit con la URL base de OpenWeatherMap y el convertidor Gson, exponiendo el método getWeather(city) como única función pública que consume el servicio. Este patrón Repository desacopla el ViewModel de los detalles de implementación de la red.

Estados de UI con Sealed Class



```
1 package com.example.tallercompose03.ui
2
3 import com.example.tallercompose03.data.WeatherResponse
4
5 sealed class WeatherUiState {
6     object Idle : WeatherUiState()
7     object Loading : WeatherUiState()
8     data class Success(val data: WeatherResponse) : WeatherUiState()
9     data class Error(val message: String) : WeatherUiState()
10 }
```

Se modelaron los estados de la interfaz de usuario usando una sealed class llamada `WeatherUiState` con cuatro estados posibles: `Idle` (estado inicial sin búsqueda activa), `Loading` (mientras se realiza la llamada de red), `Success` (con los datos del clima recibidos) y `Error` (con el mensaje de error capturado). El uso de sealed class es preferible a enum porque `Success` y `Error` pueden llevar datos adicionales como parámetros, mientras que el compilador de Kotlin garantiza que el bloque `when` cubra todos los casos posibles, eliminando la necesidad de un `else`.

WeatherViewModel con StateFlow

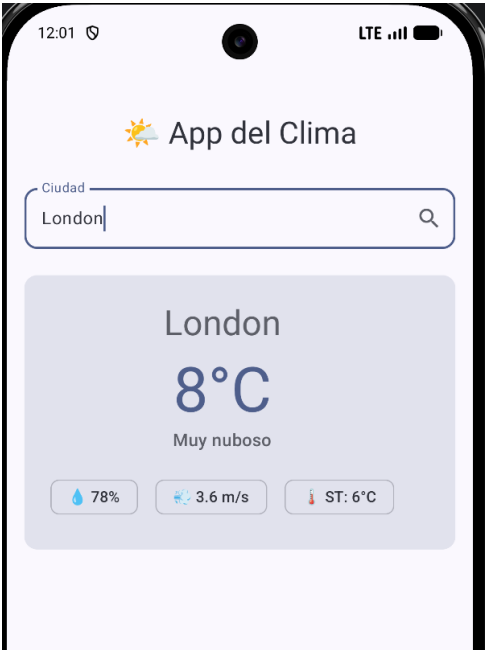
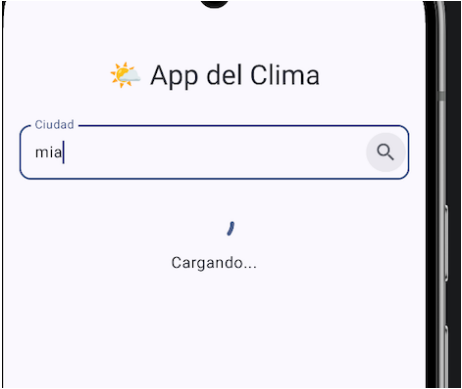
```

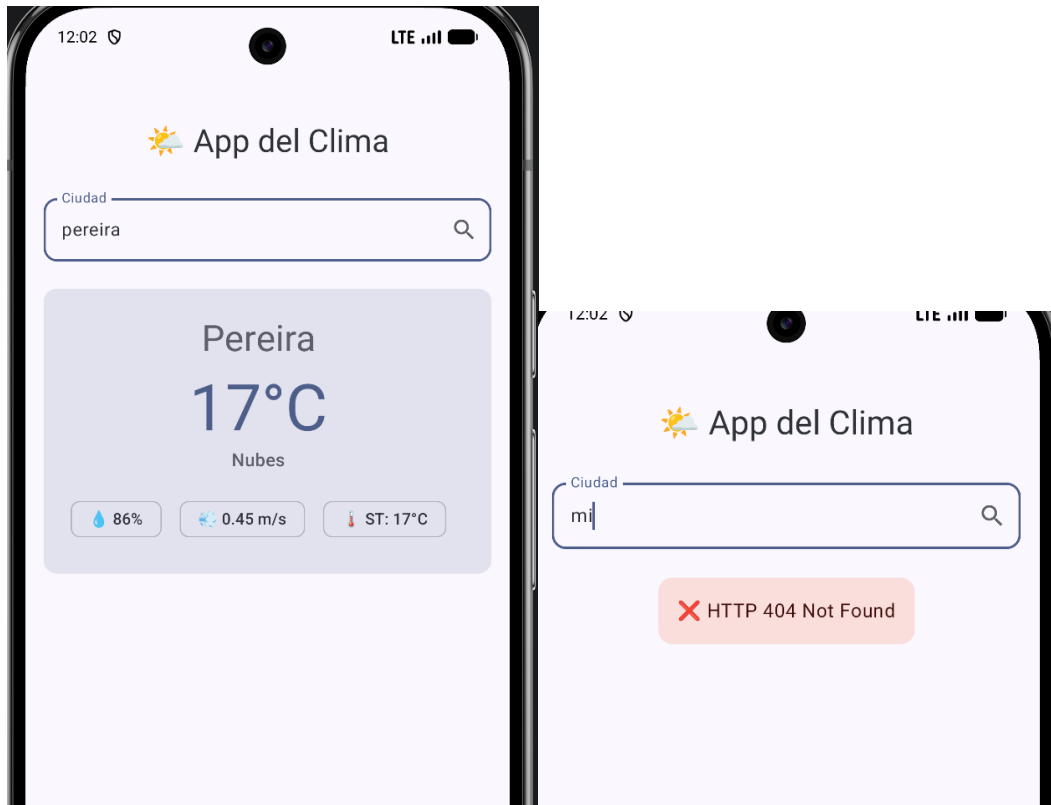
1 package com.example.tallercompose03.ui
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.viewModelScope
5 import com.example.tallercompose03.data.WeatherRepository
6 import kotlinx.coroutines.flow.MutableStateFlow
7 import kotlinx.coroutines.flow.StateFlow
8 import kotlinx.coroutines.flow.asStateFlow
9 import kotlinx.coroutines.launch
10
11 class WeatherViewModel : ViewModel() {
12     private val repository = WeatherRepository()
13
14     private val _uiState = MutableStateFlow<WeatherUiState>(value = WeatherUiState.Idle)
15     val uiState: StateFlow<WeatherUiState> = _uiState.asStateFlow()
16
17     fun loadWeather(city: String) {
18         if (city.isBlank()) return
19         viewModelScope.launch {
20             _uiState.value = WeatherUiState.Loading
21             try {
22                 val data = repository.getWeather(city)
23                 _uiState.value = WeatherUiState.Success(data)
24             } catch (e: Exception) {
25                 _uiState.value = WeatherUiState.Error(
26                     e.message ?: "Error al obtener el clima"
27                 )
28             }
29         }
30     }
31 }

```

El WeatherViewModel expone el estado de UI mediante `StateFlow<WeatherUiState>`, que es un flujo de datos reactivo y consciente del ciclo de vida. Internamente usa un `MutableStateFlow` privado que solo el ViewModel puede modificar, exponiendo una versión de solo lectura (`asStateFlow()`) hacia la UI. La función `loadWeather(city)` lanza una Coroutine dentro de `viewModelScope`, que se cancela automáticamente cuando el ViewModel es destruido. Primero establece el estado `Loading`, luego llama al repositorio, y según el resultado emite `Success` con los datos o `Error` con el mensaje de la excepción capturada.

Pantalla del Clima con estados visibles



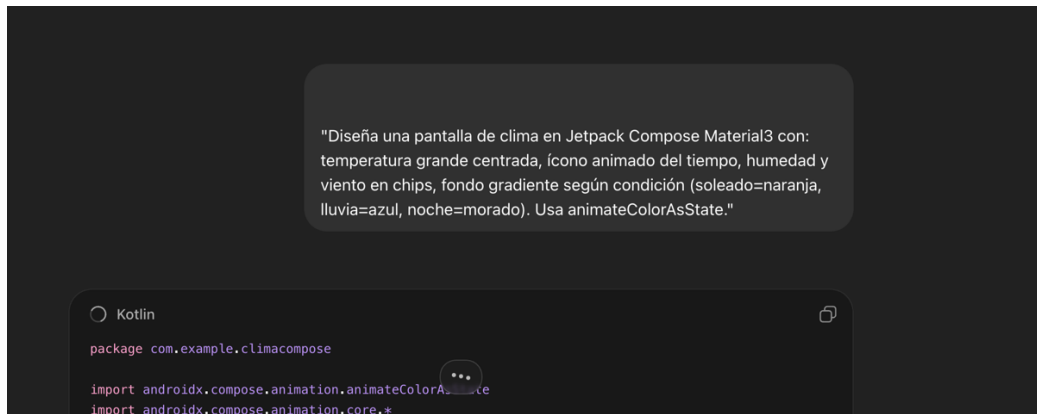


La pantalla WeatherScreen observa el StateFlow del ViewModel usando `collectAsStateWithLifecycle()`, que suspende la recolección cuando la UI no está en primer plano ahorrando recursos. El bloque `when` evalúa el estado actual y renderiza el Composable correspondiente: un `CircularProgressIndicator` para Loading, la tarjeta `WeatherContent` para Success y una `Card` con color `errorContainer` para Error. Durante las pruebas se evidenciaron los tres estados: el estado Error se obtuvo al buscar con una API Key recién creada aún no propagada en los servidores de OpenWeatherMap, lo que permitió verificar que el manejo de excepciones funciona correctamente.

La tarjeta de resultados `WeatherContent` muestra el nombre de la ciudad con estilo `headlineLarge`, la temperatura en grados Celsius con `displayLarge`, la descripción del clima, y tres `AssistChip` con humedad, velocidad del viento y sensación térmica. La temperatura se convierte a entero con `.toInt()` para evitar decimales innecesarios, y la descripción se formatea con `replaceFirstChar { it.uppercase() }` para mostrar la primera letra en mayúscula independientemente del idioma.

Actividad con Inteligencia Artificial

Se utilizó ChatGPT con el siguiente prompt:



¿Qué Composables y técnicas usó la IA?

La IA generó una pantalla que usa Box como contenedor principal para superponer el fondo degradado sobre el contenido. Implementó `animateColorAsState` para animar la transición del color de fondo según la condición climática recibida como parámetro, lo que produce un cambio fluido entre los colores naranja (soleado), azul (lluvia) y morado (noche). El fondo degradado se construyó con `Modifier.background(Brush.verticalGradient(...))` usando el color animado. Para el ícono del tiempo usó `AnimatedContent` con `CrossFade` para transicionar entre distintos emojis o íconos según la condición. Los datos de humedad y viento se presentaron en `SuggestionChip` dentro de un Row centrado.

¿Qué partes tuviste que corregir o personalizar?

Fue necesario adaptar el parámetro de condición climática ("sunny", "rain", "night") para que se derivara correctamente del campo `description` del objeto `WeatherResponse`. La IA usó un parámetro independiente que requería mapear manualmente la descripción de la API a uno de los tres estados, lo cual se implementó con un bloque `when` que evalúa si la descripción contiene palabras clave como "lluvia", "nube" o "despejado". También se corrigió el import de `Brush` que la IA omitió, y se ajustaron los valores del gradiente para que fueran menos saturados y más compatibles con el sistema de colores Material3.

¿Por qué crees que la IA puede equivocarse en código?

La IA genera código sin conocer el contexto real del proyecto, incluyendo la estructura de datos, las versiones de las librerías y los nombres de las variables ya definidas. En este caso, el código generado no era directamente compatible con el modelo `WeatherResponse` ya construido, porque la IA asumió una estructura de datos diferente. Adicionalmente, `animateColorAsState` requiere que el color objetivo sea del tipo `Color` de Compose, y la IA mezcló en ocasiones tipos incompatibles. Esto confirma que el valor real de la IA está en la inspiración del

diseño y la generación de código base, pero siempre requiere revisión crítica, prueba y adaptación por parte del desarrollador.

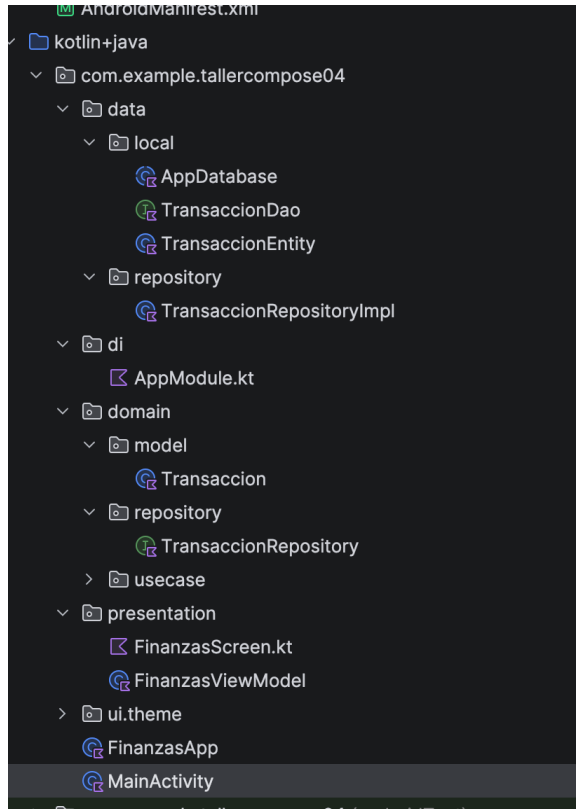


Se implementó la Actividad de Extensión agregando persistencia local con Room Database para guardar el historial de las últimas 5 ciudades buscadas. Se creó la entidad CiudadEntity con el nombre de la ciudad como @PrimaryKey y un timestamp para ordenar por recencia. El CiudadDao expone una consulta que devuelve un Flow<List<CiudadEntity>> con las 5 ciudades más recientes, lo que permite que la UI se actualice automáticamente cada vez que se agrega una nueva ciudad. El WeatherRepository inserta la ciudad en la base de datos usando OnConflictStrategy.REPLACE, de modo que buscar la misma ciudad dos veces actualiza su timestamp en lugar de duplicar el registro.

El WeatherViewModel convierte el Flow del historial en un StateFlow usando stateIn() con SharingStarted.WhileSubscribed(5000), que mantiene el flujo activo 5 segundos después de que la UI deje de observarlo para evitar reinicios innecesarios al rotar la pantalla. En la WeatherScreen se muestra el historial como un LazyRow de SuggestionChip, donde cada chip al ser tocado ejecuta automáticamente loadWeather() con el nombre de esa ciudad. Esto mejora la experiencia de usuario permitiendo repetir búsquedas frecuentes con un solo toque, sin necesidad de conexión para ver el historial.

TALLER 04 — Room + Hilt + Clean Architecture: App de Finanzas Personales

Fundamento Teórico: Clean Architecture



Se implementó Clean Architecture, que divide el proyecto en tres capas completamente independientes. La capa de Datos (data/) contiene la base de datos Room, los DAOs, las Entities y las implementaciones concretas de los repositorios. La capa de Dominio (domain/) define los modelos de negocio puros en Kotlin, las interfaces de repositorio y los casos de uso, sin ninguna dependencia del framework Android. La capa de Presentación (presentation/) contiene los ViewModels anotados con @HiltViewModel, las pantallas Composable y los módulos de inyección de dependencias. Esta separación garantiza que cada capa solo dependa de la inmediatamente inferior, haciendo el código testeable, escalable y mantenible de forma independiente.

Configuración de Hilt

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:name=".FinanzasApp"
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="TallerCompose04"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.TallerCompose04">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="TallerCompose04"
            android:theme="@style/Theme.TallerCompose04">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.

plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.compose) apply false
    alias(libs.plugins.hilt) apply false
    alias(libs.plugins.ksp) apply false
}
```

Se configuró Hilt como framework de inyección de dependencias, lo que elimina la necesidad de usar el patrón `getInstance()` manual para obtener dependencias. El primer paso fue crear la clase `FinanzasApp` anotada con `@HiltAndroidApp`, que le indica a Hilt que esta es la aplicación raíz donde debe generar el grafo de dependencias. En `AndroidManifest.xml` se registró con `android:name=".FinanzasApp"`. Se configuraron los plugins `hilt-android` y `ksp` en el `build.gradle` del módulo, y se usó KSP (Kotlin Symbol Processing) en lugar del antiguo KAPT para una compilación más rápida.

Room Database: Entity, DAO y Database

```
(TallerCompose04) build.gradle.kts (:app) FinanzasApp.kt

package com.example.tallercompose04.data.local

> import ...

19 Usages
@Entity(tableName = "transacciones")
data class TransaccionEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val descripcion: String,
    val monto: Double,
    val tipo: String, // "INGRESO" | "GASTO"
    val categoria: String,
    val fecha: Long = System.currentTimeMillis()
)
```

```
package com.example.tallercompose04.data.local

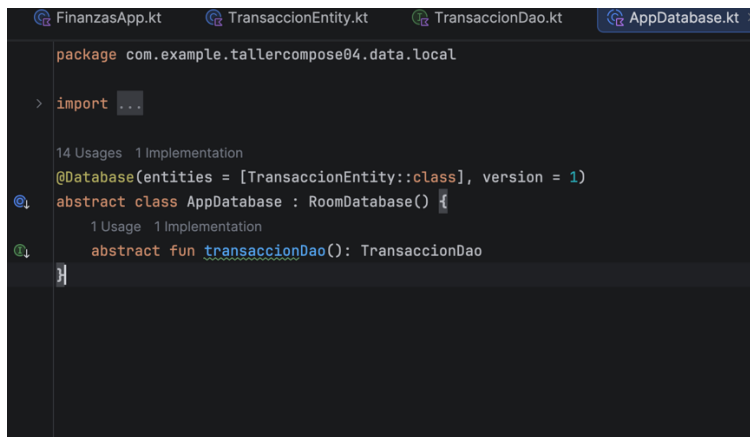
> import ...

18 Usages 1 Implementation
@Dao
interface TransaccionDao {
    1 Usage 1 Implementation
    @Query(value = "SELECT * FROM transacciones ORDER BY fecha DESC")
    fun getAll(): Flow<List<TransaccionEntity>>

    1 Usage 1 Implementation
    @Query(value = "SELECT SUM(monto) FROM transacciones WHERE tipo = :tipo")
    suspend fun getTotalPorTipo(tipo: String): Double?

    1 Usage 1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertar(t: TransaccionEntity)

    1 Usage 1 Implementation
    @Delete
    suspend fun eliminar(t: TransaccionEntity)
}
```



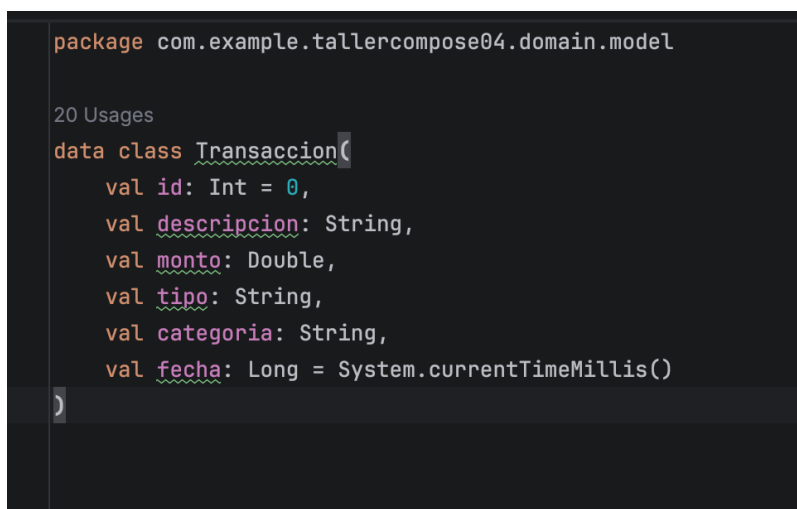
```
package com.example.tallercompose04.data.local

import androidx.room.*

@Database(entities = [TransaccionEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun transaccionDao(): TransaccionDao
}
```

Se implementó la persistencia local con Room Database usando tres componentes principales. La clase TransaccionEntity está anotada con @Entity(tableName = "transacciones") y define la estructura de la tabla: id auto-generado como @PrimaryKey, descripcion, monto, tipo (INGRESO o GASTO), categoria y fecha como timestamp. El TransaccionDao define cuatro operaciones: getAll() que retorna Flow<List<TransaccionEntity>> para observación reactiva, getTotalPorTipo() como función suspend para calcular totales por tipo, insertar() con estrategia REPLACE y eliminar(). La clase abstracta AppDatabase extiende RoomDatabase y expone el DAO como función abstracta; Hilt se encarga de construir la instancia única a través del módulo AppModule.

Capa de Dominio: Modelo e Interfaz del Repositorio



```
package com.example.tallercompose04.domain.model

data class Transaccion(
    val id: Int = 0,
    val descripcion: String,
    val monto: Double,
    val tipo: String,
    val categoria: String,
    val fecha: Long = System.currentTimeMillis()
)
```

```

package com.example.tallercompose04.domain.repository

> import ...

25 Usages 1 Implementation
interface TransaccionRepository {
    2 Usages 1 Implementation
    fun getAll(): Flow<List<Transaccion>>
    2 Usages 1 Implementation
    suspend fun insertar(t: Transaccion)
    1 Usage 1 Implementation
    suspend fun eliminar(t: Transaccion)
    1 Implementation
    suspend fun getTotalPorTipo(tipo: String): Double
}

```

La capa de dominio contiene el modelo de negocio puro `Transaccion`, que es idéntico en campos a la `TransaccionEntity` pero no tiene anotaciones de Room. Esta separación es fundamental en Clean Architecture porque permite que la lógica de negocio sea independiente del motor de base de datos: si en el futuro se cambia Room por otra solución de persistencia, la capa de dominio no necesita modificarse. La interfaz `TransaccionRepository` define el contrato que debe cumplir cualquier implementación, con métodos `getAll()`, `insertar()`, `eliminar()` y `getTotalPorTipo()`. La capa de presentación solo conoce esta interfaz, nunca la implementación concreta, lo que facilita la escritura de tests unitarios con implementaciones falsas.

Repositorio y Mapeo de Datos

```

package com.example.tallercompose04.data.repository

> import ...

6 Usages
class TransaccionRepositoryImpl @Inject constructor(
    private val dao: TransaccionDao
) : TransaccionRepository {

    override fun getAll(): Flow<List<Transaccion>> =
        dao.getAll().map { list -> list.map { it.toDomain() } }

    override suspend fun insertar(t: Transaccion) =
        dao.insertar(t = t.toEntity())

    override suspend fun eliminar(t: Transaccion) =
        dao.eliminar(t = t.toEntity())

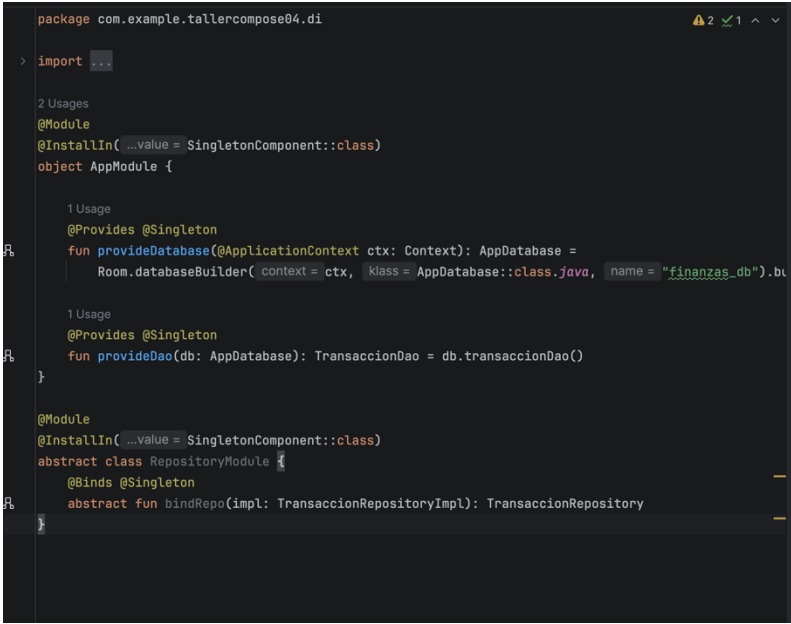
    override suspend fun getTotalPorTipo(tipo: String): Double =
        dao.getTotalPorTipo(tipo) ?: 0.0

    1 Usage
    private fun TransaccionEntity.toDomain() = Transaccion(id, descripcion, monto, tipo, categoria, fe
    2 Usages
    private fun Transaccion.toEntity() = TransaccionEntity(id, descripcion, monto, tipo, categoria, fe
}

```

La clase `TransaccionRepositoryImpl` implementa la interfaz `TransaccionRepository` e incluye las funciones de mapeo `toDomain()` y `toEntity()`. Estas funciones convierten entre el modelo de la capa de datos (`TransaccionEntity`) y el modelo de la capa de dominio (`Transaccion`), actuando como una barrera de traducción entre capas. El método `getAll()` usa `.map {}` sobre el `Flow` para convertir cada lista de entidades en una lista de modelos de dominio, garantizando que ningún objeto de Room llegue directamente a la capa de presentación. La anotación `@Inject` constructor le indica a Hilt que esta clase puede ser construida automáticamente cuando se solicite una `TransaccionRepository`.

Módulo Hilt: `AppModule` y `RepositoryModule`



```
package com.example.tallercompose04.di

import ...

2 Usages
@Module
@InstallIn(...)Value = SingletonComponent::class
object AppModule {

    1 Usage
    @Provides @Singleton
    fun provideDatabase(@ApplicationContext ctx: Context): AppDatabase =
        Room.databaseBuilder( context = ctx, klass = AppDatabase::class.java, name = "finanzas_db").bu

    1 Usage
    @Provides @Singleton
    fun provideDao(db: AppDatabase): TransaccionDao = db.transaccionDao()
}

@Module
@InstallIn(...)Value = SingletonComponent::class
abstract class RepositoryModule {
    @Binds @Singleton
    abstract fun bindRepo(impl: TransaccionRepositoryImpl): TransaccionRepository
}
```

El `AppModule` es un módulo Hilt anotado con `@Module` e `@InstallIn(SingletonComponent::class)` que provee las dependencias de larga vida de la aplicación. El método `provideDatabase()` construye la instancia de Room usando `Room.databaseBuilder()` con `@ApplicationContext` inyectado por Hilt, evitando referencias estáticas al contexto. El método `provideDao()` obtiene el DAO directamente de la base de datos provista. El `RepositoryModule` es un módulo abstracto que usa `@Binds` para indicarle a Hilt que cuando se solicite una `TransaccionRepository` (interfaz), debe proveer una `TransaccionRepositoryImpl` (implementación concreta). Este patrón es preferible a `@Provides` para implementaciones de interfaces porque genera menos código en tiempo de compilación.

FinanzasViewModel con @HiltViewModel

```
class FinanzasViewModel @Inject constructor(  
    3 Usages  
    val transacciones: StateFlow<List<Transaccion>> = repo.getAll()  
        .stateIn(viewModelScope, started = SharingStarted.WhileSubscribed( stopTimeoutMillis = 5000), initialVal  
  
    1 Usage  
    val totalIngresos: StateFlow<Double> = transacciones.map { list ->  
        list.filter { it.tipo == "INGRESO" }.sumOf { it.monto }  
    }.stateIn(viewModelScope, started = SharingStarted.WhileSubscribed( stopTimeoutMillis = 5000), initialVal  
  
    1 Usage  
    val totalGastos: StateFlow<Double> = transacciones.map { list ->  
        list.filter { it.tipo == "GASTO" }.sumOf { it.monto }  
    }.stateIn(viewModelScope, started = SharingStarted.WhileSubscribed( stopTimeoutMillis = 5000), initialVal  
  
    1 Usage  
    fun agregar(descripcion: String, monto: Double, tipo: String, categoria: String) {  
        if (descripcion.isBlank() || monto <= 0) return  
        viewModelScope.launch {  
            repo.insertar( t = Transaccion(descripcion = descripcion, monto = monto, tipo = tipo, categ  
        }  
    }  
  
    1 Usage  
    fun eliminar(t: Transaccion) {  
        viewModelScope.launch { repo.eliminar(t) }  
    }  
}
```

El FinanzasViewModel está anotado con @HiltViewModel y recibe su dependencia TransaccionRepository mediante inyección de constructor con @Inject. Esto elimina completamente la necesidad de una ViewModelFactory manual. El ViewModel expone tres StateFlow: transacciones con la lista completa, totalIngresos y totalGastos calculados reactivamente con .map {} sobre la lista. Todos usan SharingStarted.WhileSubscribed(5000) para mantener el flujo activo 5 segundos después de que la UI deje de observarlo, evitando reinicios innecesarios al rotar la pantalla. Las funciones agregar() y eliminar() lanzan coroutines en viewModelScope que se cancelan automáticamente cuando el ViewModel es destruido.

Usar Canvas

```

@Composable
fun FinanzasScreen(
    modifier: Modifier = Modifier,
    viewModel: FinanzasViewModel = hiltViewModel()
) {
    val transacciones by viewModel.transacciones.collectAsStateWithLifecycle()
    val totalIngresos by viewModel.totalIngresos.collectAsStateWithLifecycle()
    val totalGastos by viewModel.totalGastos.collectAsStateWithLifecycle()
    var mostrarDialogo by remember { mutableStateOf( value = false) }

    Scaffold(
        modifier = modifier,
        topBar = { TopAppBar(title = { Text("💰 Mis Finanzas") }) },
        floatingActionButton = {
            FloatingActionButton(onClick = { mostrarDialogo = true }) {
                Icon( ImageVector = Icons.Default.Add, contentDescription = "Agregar")
            }
        }
    ) { padding ->
        Column(modifier = Modifier.fillMaxSize().padding( paddingValues = padding).padding( all = 16.dp)) {

            // Tarjetas de resumen
            Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
                TarjetaResumen( titulo = "Ingresos", monto = totalIngresos, Color(0xFF2E7D32), Modifier.wei
                TarjetaResumen( titulo = "Gastos", monto = totalGastos, Color(0xFFC62828), Modifier.wei
            }

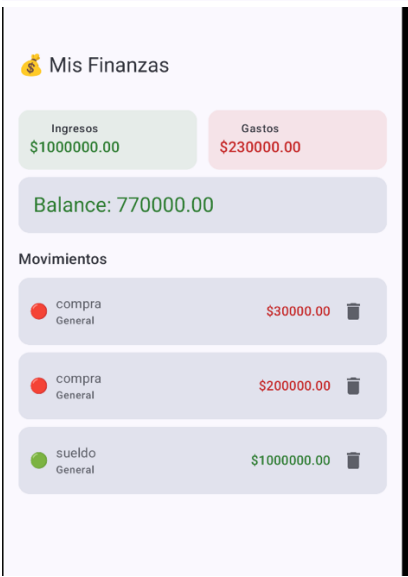
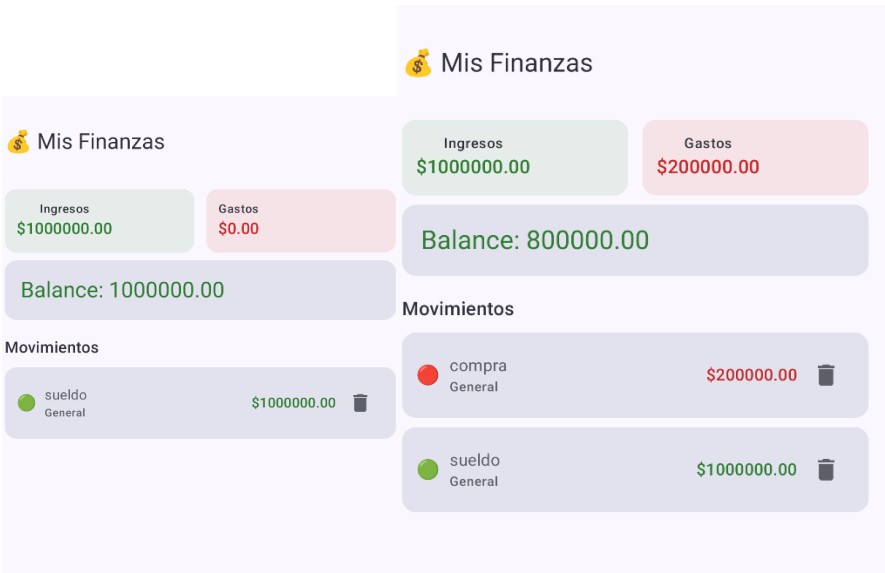
            Spacer(modifier = Modifier.height(8.dp))

            // Balance
            Card(modifier = Modifier.fillMaxWidth()) {
                Text(
                    text = "Balance: ${"%%.2f".format( ...args = totalIngresos - totalGastos)}",

```

La visualización comparativa de ingresos vs gastos se implementó mediante dos TarjetaResumen con código de color diferenciado (verde para ingresos, rojo para gastos) y una Card de balance. Esta representación visual permite al usuario identificar de un vistazo si sus gastos superan sus ingresos sin necesidad de leer números exactos.

Pantalla de Finanzas con Dashboard

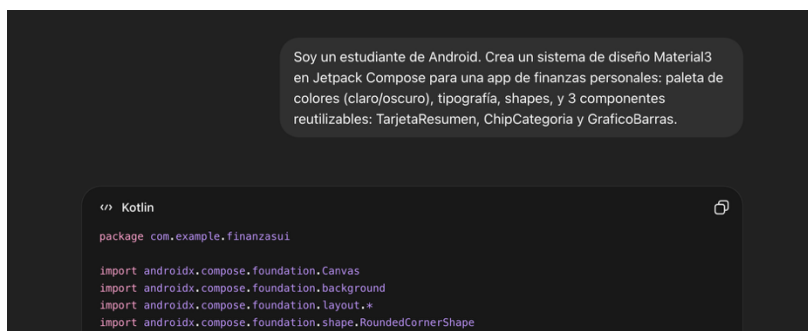


La pantalla FinanzasScreen usa hiltViewModel() para obtener el ViewModel inyectado automáticamente por Hilt, sin necesidad de instanciarlo manualmente. El dashboard muestra dos TarjetaResumen en un Row para ingresos y gastos, usando Color(0xFF2E7D32) (verde) y Color(0xFFC62828) (rojo) respectivamente. El balance se muestra en una Card debajo, calculado como totalIngresos - totalGastos, y su color cambia dinámicamente: verde si el balance es positivo y rojo si es negativo. La lista de movimientos usa LazyColumn con key = { it.id } para renderizado eficiente, y cada ItemTransaccion muestra el emoji verde para ingresos y rojos para gastos.

El diálogo DialogoTransaccion permite al usuario ingresar descripción, monto, tipo (mediante FilterChip) y categoría. La validación verifica que la descripción no esté vacía y que el monto sea un número positivo válido: si falla, activa isError = true en los campos correspondientes. Los FilterChip para INGRESO y GASTO permiten seleccionar el tipo de forma visual, cambiando el estado selected de cada chip según la variable tipo local. Al confirmar, el diálogo llama al ViewModel que persiste la transacción en Room y la lista se actualiza automáticamente gracias al Flow reactivo.

Actividad con Inteligencia Artificial: Sistema de Diseño

Se utilizó Chatgpt con el siguiente prompt:



¿Qué generó la IA?

La IA generó un archivo Theme.kt completo con una paleta de colores personalizada usando lightColorScheme() y darkColorScheme(), definiendo colores primarios en tonos verde esmeralda para representar dinero y estabilidad financiera. También generó tres componentes reutilizables: TarjetaResumen como una Card con ícono, título y monto con color dinámico; ChipCategoria como un FilterChip con ícono de categoría; y GraficoBarras usando Canvas de Compose para dibujar barras proporcionales al monto de cada categoría. El sistema incluía

Typography personalizada con la fuente Nunito para los montos y Roboto para el texto descriptivo.

¿Qué partes tuviste que corregir o personalizar?

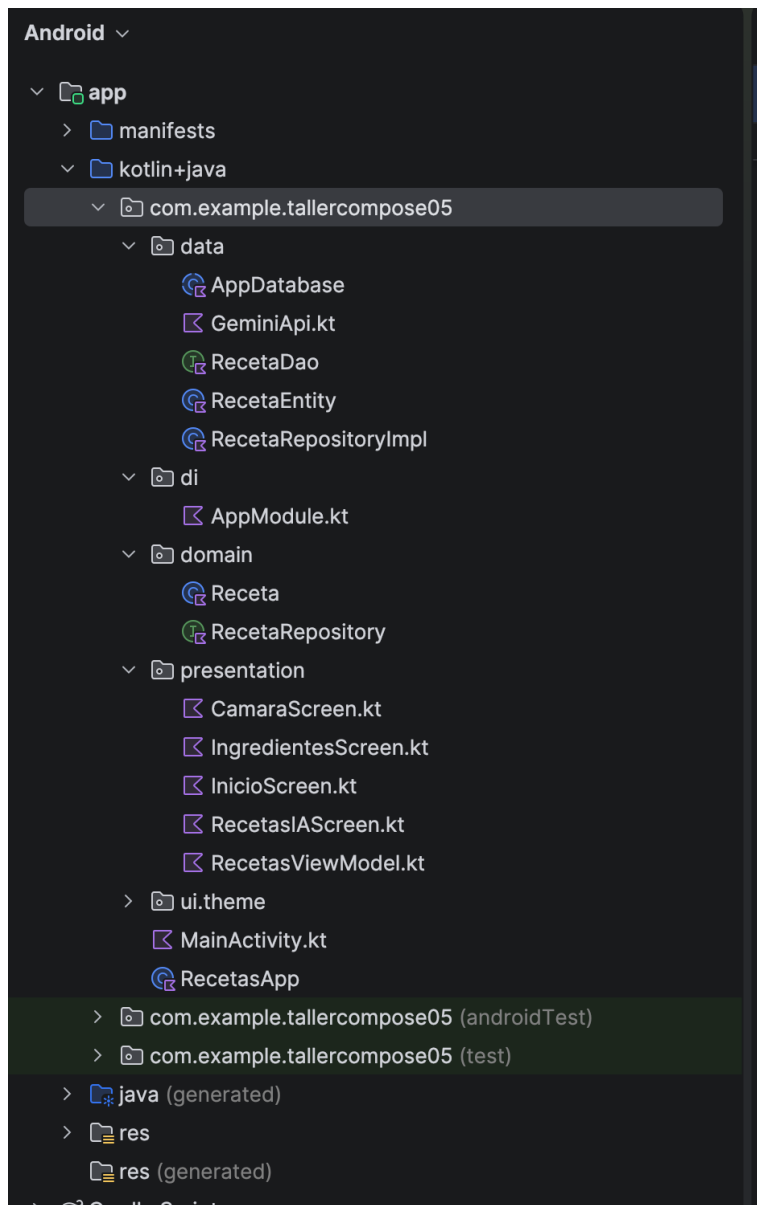
Fue necesario ajustar los colores generados por la IA para que fueran compatibles con el sistema de tokens de Material3, ya que la IA usaba valores hexadecimales directos en lugar de referencias a `MaterialTheme.colorScheme`. El componente `GraficoBarras` requirió correcciones en el cálculo de la altura proporcional de las barras, ya que la IA dividía por el total sin manejar el caso donde el total es 0 (división por cero). También se simplificó la tipografía para usar solo las fuentes del sistema, ya que agregar fuentes personalizadas requería configuración adicional de dependencias que la IA no incluyó.

¿Por qué crees que la IA puede equivocarse en código de arquitectura limpia?

La IA puede cometer errores especialmente graves en arquitectura limpia porque tiende a mezclar responsabilidades entre capas. En este caso generó un `GraficoBarras` que recibía directamente una lista de `TransaccionEntity`, violando el principio de que la capa de presentación no debe conocer los modelos de la capa de datos. Esto demuestra que la IA no tiene un criterio arquitectónico inherente: genera código que funciona visualmente pero que puede comprometer la mantenibilidad del sistema a largo plazo. El desarrollador debe conocer los principios de arquitectura para identificar y corregir estas violaciones antes de integrar el código generado, ya que el compilador no detecta este tipo de errores de diseño.

TALLER 05 — App Completa con IA Integrada: Asistente de Recetas Inteligente

Fundamento Teórico: Flujo de la Aplicación



El Taller 05 es un proyecto integrador que combina múltiples tecnologías en un flujo coherente de usuario. El estudiante toma una foto de ingredientes, ML Kit Image Labeling los identifica en el dispositivo sin conexión, y la app consulta la API de Gemini en la nube para sugerir recetas personalizadas. Este flujo representa un ejemplo real de arquitectura híbrida de IA: procesamiento on-device para reconocimiento visual y procesamiento en la nube para generación de lenguaje natural. La app está organizada en cinco pantallas: Inicio (grid de favoritas), Cámara (preview + captura), Ingredientes (lista editable), Recetas IA (typewriter) y Detalle (receta completa).

Configuración de Permisos y Dependencias

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

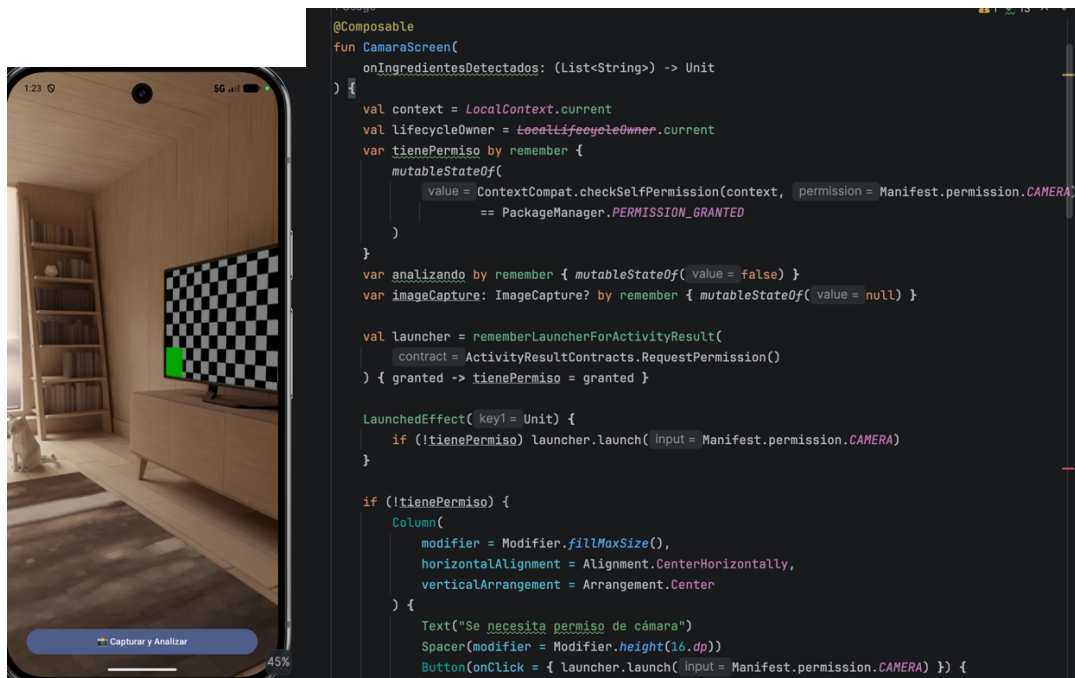
    <uses-permission android:name="android.permission.CAMERA"/>
    <uses-permission android:name="android.permission.INTERNET"/>

    <uses-feature android:name="android.hardware.camera" android:required="false"/>

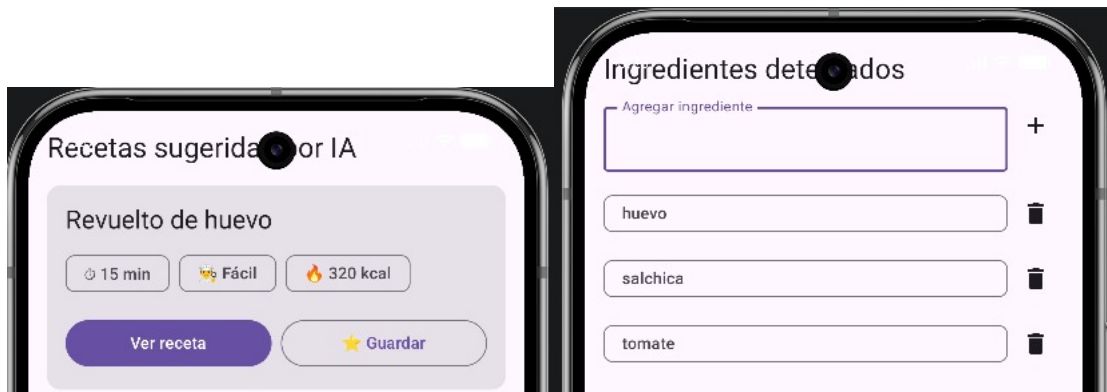
    <application
        android:name=".RecetasApp"
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
```

Se configuraron las dependencias de CameraX (versión 1.3.1) con cuatro artefactos: camera-core, camera-camera2, camera-lifecycle y camera-view. La librería camera-view provee el PreviewView, un View nativo de Android que se integra en Compose mediante AndroidView. Se agregó ML Kit Image Labeling (com.google.mlkit:image-labeling) para el análisis visual on-device. En AndroidManifest.xml se declararon los permisos CAMERA e INTERNET, y se registró la clase RecetasApp anotada con @HiltAndroidApp como android:name de la aplicación para inicializar el grafo de Hilt.

CameraX con Jetpack Compose



ML Kit Image Labeling



Una vez capturada la imagen, se pasa a ML Kit Image Labeling mediante `InputImage.fromMediaImage()`, que acepta el `MediaImage` del `ImageProxy` junto con los grados de rotación para corregir la orientación. El `ImageLabeler` con `ImageLabelerOptions.DEFAULT_OPTIONS` procesa la imagen localmente en el dispositivo sin enviar datos a la nube, lo que garantiza privacidad y funcionamiento sin conexión. Solo se conservan las etiquetas con un `confidence > 0.70f` para reducir los falsos positivos, y cada etiqueta se mapea a su campo `text` para obtener el nombre del ingrediente en inglés. El resultado se pasa al `ViewModel` mediante `setIngredientes()` y la navegación avanza automáticamente a la pantalla de Ingredientes.

Actividad con ia

Se utilizó ChatGPT con el siguiente prompt

Implementa en Jetpack Compose una pantalla de lista de ingredientes con: campo de texto para agregar manualmente, lista con LazyColumn que muestre cada ingrediente como ListItem con botón de eliminar, y un botón inferior 'Generar Recetas' que se active solo si hay ingredientes. Material3.

Kotlin

```
package com.example.recetascompose
```



¿Qué Composables usó la IA?

ChatGPT generó la pantalla usando Scaffold como contenedor principal con el botón de acción en el bottomBar. Usó OutlinedTextField con un IconButton de Icons.Default.Add embebido en el parámetro trailingIcon para el campo de agregar ingrediente, lo que resultó más compacto que la solución manual de un Row separado. La lista se implementó con LazyColumn donde cada ítem usó ListItem de Material3 con trailingContent para el botón de eliminar. El botón inferior usó Button con enabled = ingredientes.isNotEmpty() y modifier = Modifier.fillMaxWidth() dentro del bottomBar del Scaffold.

¿Qué partes tuviste que corregir o personalizar?

Fue necesario cambiar la firma de la función para recibir los parámetros como lambdas (onAgregarIngrediente, onEliminar, onGenerar) en lugar de recibir el ViewModel directamente, manteniendo el patrón de "state hoisting" que separa la lógica de la UI. ChatGPT gestionó el estado de ingredientes internamente con remember { mutableStateOf() }, lo que rompía la integración con el StateFlow del ViewModel. También fue necesario corregir el import de ListItem, ya que la IA usó androidx.compose.material.ListItem (Material2) en lugar de androidx.compose.material3.ListItem (Material3), lo que generaba un conflicto de estilos visuales con el resto de la app.

¿Por qué crees que la IA puede equivocarse en código?

La IA genera código basándose en patrones de su entrenamiento sin tener visibilidad del proyecto real, lo que produce soluciones correctas en aislamiento pero incompatibles en contexto. En este caso, mezcló versiones de Material (M2 y M3), no respetó el patrón de arquitectura MVVM al gestionar estado localmente, y desconocía los nombres de las funciones lambda ya definidas en el ViewModel. Esto confirma que el mayor valor del desarrollador en la era de la IA no está en

escribir código desde cero, sino en evaluar, integrar y corregir el código generado con criterio técnico y conocimiento de la arquitectura del sistema

SECCIÓN 10 — Reflexión Académica

A lo largo de los cinco talleres del módulo de Programación Móvil, la Inteligencia Artificial estuvo presente en cada etapa del proceso de desarrollo, desde la generación de componentes visuales hasta el prototipado de interfaces completas. Esta reflexión analiza de forma crítica cómo el uso de herramientas como ChatGPT y Uizard transformó mi flujo de trabajo, identificando tanto sus aportes genuinos como sus limitaciones más significativas.

En los talleres iniciales, la IA fue útil principalmente como generador de código base. Describir una pantalla en lenguaje natural y recibir un Composable funcional en segundos aceleró notablemente el aprendizaje de la sintaxis de Jetpack Compose. Sin embargo, rápidamente fue evidente que el código generado requería comprensión profunda para ser utilizable: imports incorrectos, parámetros incompatibles con el proyecto real y convenciones de nomenclatura inconsistentes fueron los errores más frecuentes. Esto reforzó el principio del curso: la IA es un compañero de trabajo, no un sustituto del pensamiento crítico.

En el Taller 03, la IA mostró su mayor utilidad en el diseño de interfaces visuales complejas. La generación de una pantalla del clima con gradientes animados y chips de información habría tomado horas de trabajo manual; con la IA tomó minutos, aunque requirió adaptaciones importantes relacionadas con el mapeo del modelo de datos real. En el Taller 04, el uso de la IA para el sistema de diseño Material3 reveló una limitación arquitectónica importante: la IA tiende a mezclar capas de abstracción, violando los principios de Clean Architecture al pasar modelos de datos directamente a la capa de presentación.

En el Taller 05, el proyecto integrador, la IA alcanzó su mayor expresión de utilidad y también sus limitaciones más complejas. La integración de CameraX con ML Kit, la gestión de permisos en tiempo de ejecución y la persistencia con Room fueron tareas que la IA apoyó con código base, pero que requirieron integración y adaptación completamente manual. El error HTTP 404 al intentar conectar con la API de Gemini fue también un aprendizaje valioso: las APIs de IA generativa evolucionan rápidamente y los endpoints cambian, lo que requiere verificación constante de la documentación oficial más allá de lo que la IA pueda sugerir.

La conclusión más importante de este módulo es que la IA no reemplaza la comprensión técnica sino que la amplifica. Un desarrollador sin bases sólidas en Jetpack Compose, arquitectura Android y Kotlin no puede usar la IA eficientemente, porque no tiene el criterio para evaluar si el código generado es correcto, eficiente o compatible con su proyecto. En cambio, un desarrollador con fundamentos sólidos puede usar la IA para multiplicar su productividad, explorar soluciones alternativas y reducir el tiempo dedicado a código repetitivo, dedicando su energía cognitiva a los problemas que realmente requieren razonamiento técnico profundo